

Architecture of Massively Scalable Applications, Spring 2026

Milestone 2: E-Commerce (Amazon) Platform
Polyglot Persistence, Authentication, Caching & Design Patterns
Deadline is Saturday 02/05/2026 at 11:59 PM

Contents

1	Overview	4
2	Git Workflow — Feature Development	5
3	Design Patterns	7
3.1	The 7 Required Patterns	7
3.2	Strategy Pattern — Refund Logic (S5-F12)	7
3.3	Observer Pattern — Event Logging	8
3.4	Chain of Responsibility — JWT Filter Chain	9
3.5	Builder Pattern — Dashboard DTOs	10
3.6	Singleton Pattern — JwtConfigurationManager	11
3.7	Factory Pattern — Event Creation	12
3.8	Adapter Pattern — NoSQL Result → DTO Conversion	13
3.9	Grading Summary	14
4	M1 Modifications Required	14
4.1	User Entity — Password Hashing	14
4.2	User Entity — Role Values (Additive)	14
4.3	Existing M1 Endpoints — JWT Authentication	15
4.4	Existing M1 Read Endpoints — Redis Caching (Explicit Enumeration)	16
4.4.1	M1 Feature GET Endpoints That Must Be Cached (Amazon: 27 total)	16
4.4.2	CRUD Baseline Endpoints That Must Be Cached	16
4.4.3	Total	17
4.4.4	Endpoints That Must Invalidate Caches (Writes)	17
4.4.5	Cache Key Convention	18
4.4.6	Cache Invalidation Strategy (Wildcard Deletion)	18
4.5	Design Pattern Retrofits to M1	19
4.6	Transaction.transactionDetails — Additive Refund Keys	21
4.7	Summary Checklist	21
5	Authentication & Authorization	22
5.1	Overview	22

5.2	JWT Token	22
5.3	Roles (Additive — Preserve M1 Values)	22
5.4	Security Configuration (Per Service)	23
5.5	Password Handling	23
6	Database Architecture	23
6.1	Six Databases (Versions Match Task 2 Exactly)	23
6.2	Memory Limitations (Required)	23
6.3	Dependency Model	24
6.4	Docker Compose Additions	24
6.5	Reference application.yml Fragments	25
7	New Entity Models	27
7.1	MongoDB Document Entities (All Services)	27
7.1.1	Common MongoEvent Interface	27
7.1.2	AuthEvent (User Service)	27
7.1.3	ProductEvent (Product Service)	27
7.1.4	OrderEvent (Order Service)	28
7.1.5	ShipmentEvent (Shipping Service)	28
7.1.6	TransactionAuditEvent (Billing Service)	28
7.2	Elasticsearch Document (Product Service)	29
7.2.1	ProductSearchDocument	29
7.3	Neo4j Entities (Order Service)	29
7.3.1	ProductNode	30
7.3.2	BOUGHT_TOGETHER Relationship	30
7.4	Cassandra Entity (Shipping Service)	30
7.4.1	ShipmentTrackingEvent	30
8	Caching Strategy	30
8.1	TTL Guidelines	30
8.2	Invalidation	31
9	Cross-Cutting Requirements	31
9.1	CC-1 — JWT on All Endpoints	31
9.2	CC-2 — Role Management	31
9.3	CC-3 — Redis Caching on M1 Endpoints	32
9.4	CC-4 — Design Pattern Implementation	33
9.5	CC-5 — Docker Compose with 6 Databases	33
9.6	CC-6 — Application Configuration (application.yml)	33
10	Features	34
10.1	User Service Features (port 8081)	34
10.1.1	[S1-F10] Register User	34
10.1.2	[S1-F11] Login	35

10.1.3	[S1-F12] Get User Activity Feed	36
10.2	Product Service Features (port 8082)	37
10.2.1	[S2-F10] Full-Text Product Search	37
10.2.2	[S2-F11] Index Product for Search	38
10.2.3	[S2-F12] Get Product Catalog Dashboard	38
10.3	Order Service Features (port 8083)	39
10.3.1	[S3-F10] Get Order Analytics Dashboard	39
10.3.2	[S3-F11] Record Product Co-Purchase	40
10.3.3	[S3-F12] Get “Customers Also Bought” Recommendations	42
10.4	Shipping Service Features (port 8084)	42
10.4.1	[S4-F10] Get Shipping Analytics Dashboard	42
10.4.2	[S4-F11] Record Shipment Tracking Event	43
10.4.3	[S4-F12] Get Shipment Tracking Timeline	44
10.5	Billing Service Features (port 8085)	45
10.5.1	[S5-F10] Get Category Revenue with Return Impact	45
10.5.2	[S5-F11] Get Transaction Lifecycle Audit	46
10.5.3	[S5-F12] Process Partial Item Refund	48

1 Overview

This milestone is worth **15%** of your final grade. You will extend the E-Commerce Platform you built in Milestone 1 by adding **authentication**, **five NoSQL databases**, **caching**, and **design patterns**. Your existing 5 services, PostgreSQL database, and 45 features remain intact but require modifications (see Section 4) — Milestone 2 adds new capabilities on top of them.

What you are adding (four pillars):

- a) **Design Patterns** — 7 design patterns from Lab 7 applied at well-defined locations across your codebase (Strategy, Observer, Chain of Responsibility, Builder, Singleton, Factory, Adapter). See Section 3.
- b) **Authentication & Authorization** — JWT-based authentication with BCrypt password hashing and role-based access control. The User Service becomes the auth provider; all other services validate tokens independently.
- c) **NoSQL Databases** — MongoDB, Redis, Elasticsearch, Neo4j, and Cassandra. Each database serves a specific architectural role.
- d) **Caching** — Redis-based caching on all read-heavy endpoints, including your existing Milestone 1 endpoints.

You will also modify your existing M1 codebase (see Section 4) to integrate password hashing, JWT protection, caching, and design pattern retrofits.

Technologies (new in M2): Spring Security, JWT (JJWT), BCrypt, MongoDB, Redis, Elasticsearch, Neo4j, Cassandra, Spring Data for each database.

Spring Boot 4.0.3 on JDK 25 (Docker base image: `eclipse-temurin:25.0.2_10-jdk`). Jackson dual dependency: Jackson 3.x (`tools.jackson.*`) for Spring Boot, Jackson 2.x (`com.fasterxml.*`) for Hibernate 7.2's JSONB FormatMapper only.

Services (unchanged from M1):

- a) **User Service** (port 8081) — Now also the authentication provider
- b) **Product Service** (port 8082) — Now with full-text search via Elasticsearch
- c) **Order Service** (port 8083) — Now with co-purchase recommendation graph via Neo4j
- d) **Shipping Service** (port 8084) — Now with time-series tracking via Cassandra
- e) **Billing Service** (port 8085) — Now with detailed transaction analytics via MongoDB

Deliverables: 15 new features (3 per service), JWT authentication on all endpoints, Redis caching on read endpoints, Docker Compose with 6 database containers, continued Git workflow from M1.

Important: This is **not** a microservices milestone. Your services still share a single PostgreSQL database. Inter-service communication comes in Milestone 3. The NoSQL databases are **additive specialized tools**, not replacements for PostgreSQL.

M1 seed data remains valid (additive modifications only): M2 does not alter any M1 PostgreSQL entity schema. The additive M1 modifications M2 introduces are:

- a) **BCrypt password hashing** on the User entity's existing `password` field (see Section 4.1) — the column type is unchanged; only how values are stored changes.
- b) **Additive refund keys** in `Transaction.transactionDetails` JSONB (see Section 4.6). M2's S5-F12 Partial Item Refund feature writes new keys (`refundAmount`, `refundedItems`, `refundStrategy`, `refundedAt`) on top of the existing M1 JSONB schema. Pre-M2 Transactions without these keys are unaffected.

All existing M1 seed rows, tests, and fixtures continue to work after you apply the M1 modifications in Section 4.

Config format (properties → yaml): M1 used `application.properties` per service. In M2 you must migrate each service's configuration to `application.yml`. The auto-grader expects YAML format for M2. Reference fragments are provided in Section 6.5.

2 Git Workflow — Feature Development

The same workflow from Milestone 1 continues. Feature IDs are now **F10–F12** per service:

- a) Start from main: `git checkout main && git pull origin main`
- b) Create feature branch: `git checkout -b feat/<service>/S{n}-F{m}/<YOUR-STUDENT-ID>`
- c) Implement: repository methods → service logic → controller endpoint → test via Postman.
- d) Commit: `git commit -m "feat(<service-name>): <description> (<YOUR-STUDENT-ID>)"`
- e) Push and create a Pull Request on GitHub.
- f) At least 1 teammate reviews and approves.
- g) **Merge into main using a regular merge commit (“Create a merge commit” on GitHub).** **Do NOT use squash merge.** A regular merge preserves the branch name in the merge commit message, which the auto-grader uses to verify who implemented which feature.
- h) **Do NOT delete the feature branch after merging.** The auto-grader will verify that each feature has a correctly-named branch containing the student ID. If you delete the branch, the grader will not be able to verify it and this may result in deductions in your grade.

Example branches for M2 features:

- `feat/user/S1-F10/55-8078` — Register User
- `feat/product/S2-F10/55-8079` — Product Search
- `feat/order/S3-F11/55-8080` — Record Product Co-Purchase
- `feat/shipping/S4-F12/55-8081` — Shipment Tracking Timeline
- `feat/billing/S5-F12/55-8082` — Partial Item Refund

Branch naming convention.

Format: `<type>/<scope>/<descriptor>/<studentID>`

- **type** — one of: `feat`, `fix`, `hotfix`, `refactor`, `docs`, `test`, `chore`, `perf`.
- **scope** — the service or area being touched: `user`, `product`, `order`, `shipping`, `billing` (for per-service work), or `m1`, `cc`, `infra` (for cross-cutting work).
- **descriptor** — the stable ID when the change maps to one (e.g., `S3-F11`, `AM-3`, `CC-2`), or a short kebab-case otherwise (e.g., `refund-rounding-bug`).
- **studentID** — your numeric student ID, always the last segment; used by the grader.

Examples:

- `feat/order/S3-F11/55-8080` — new feature (M2)
- `feat/m1/AM-3/55-8080` — M1 amendment

- `feat/cc/CC-5/55-8080` — cross-cutting requirement
- `fix/billing/refund-amount-rounding/55-8080` — bug fix
- `hotfix/user/token-expiry-leak/55-8080` — urgent fix on an already-merged branch
- `refactor/product/extract-search-service/55-8080` — internal cleanup, no behavior change

Commit message convention (Conventional Commits).

Format: `<type>(<scope>): <subject> (<studentID>)`

- **type** — same set as branch.
- **scope** — same as branch.
- **subject** — imperative mood (“add”, not “added”); no trailing period; keep under 72 characters.
- **studentID** — in parentheses at the end, so every commit is attributable.

Examples:

- `feat(order): S3-F11 record product co-purchase graph (55-8080)`
- `feat(m1): AM-3 apply JWT filter to all M1 endpoints (55-8080)`
- `feat(cc): CC-2 role management endpoint (55-8080)`
- `fix(billing): correct refund amount rounding (55-8080)`
- `refactor(user): extract token validation to helper (55-8080)`
- `test(product): add integration test for S2-F11 auto-index (55-8080)`
- `docs(cc): clarify CC-5 healthcheck timings (55-8080)`
- `chore(infra): bump postgres image to 17.4 (55-8080)`

Type quick reference.

Type	When to use
feat	New feature, amendment, cross-cutting requirement, or design-pattern implementation.
fix	Bug fix in existing code (wrong behavior, wrong value, wrong status code).
hotfix	Urgent fix on an already-merged branch — use sparingly.
refactor	Code change that neither fixes a bug nor adds a feature (renaming, extracting methods).
docs	Documentation-only change (README, comments, this spec).
test	Tests only (added or fixed).
chore	Build, dependency, or tooling change with no runtime impact.
perf	Performance improvement without behavior change.

Rules:

- One branch → one PR → one work unit. Don’t bundle unrelated changes.
- Always end the branch name and commit subject with your student ID in the exact format shown above — the auto-grader matches on it.
- Prefer the stable IDs (`S<n>-F<m>`, `AM-<n>`, `CC-<n>`) as the descriptor when the change maps to one. Use a kebab-case slug only for ad-hoc fixes or refactors with no predefined ID.
- Where a design pattern (DP-1..DP-7) is implemented across multiple branches, cite the DP ID in each commit message (e.g., “implements DP-2 Observer”).

Important: Any team member who has no commits in the repository, or whose commits cannot be matched to any feature branch, will receive a ***ZERO*** as a result.

Important Note: If any member missed any of the git checks (like pushed some change directly on the main and not creating a PR for it) or any other thing, it will get reflected on everyone in the team and not that member only. Same goes for any test case or any check in the entire project.

3 Design Patterns

M2 requires you to apply **7 design patterns** from Lab 7 at specific, well-defined locations in your codebase.

3.1 The 7 Required Patterns

#	Pattern	Category	Where Applied
1	Strategy	Behavioral	S5-F12 refund logic
2	Observer	Behavioral	MongoDB event logging across all services
3	Chain of Responsibility	Behavioral	JWT authentication filter chain
4	Builder	Creational	Dashboard and analytics DTOs
5	Singleton	Creational	JwtService / ConfigurationManager
6	Factory	Creational	MongoDB event object creation
7	Adapter	Structural	NoSQL query result → DTO conversion

Distribution: 3 Creational, 1 Structural, 3 Behavioral — balanced across all 3 pattern categories.

3.2 Strategy Pattern — Refund Logic (S5-F12)

Purpose: The S5-F12 partial item refund feature has multiple business rules that must each be encapsulated as a separate strategy (full-transaction refund, selected-items-only refund, no-refund when the return window has expired). Replace if-else chains in the service with a Strategy selector.

Structure:

- RefundStrategy interface with `calculateRefund(Transaction transaction, RefundRequest request)` method returning a `RefundResult` (amount + reason code + list of refunded item IDs)
- **3 concrete strategies (required):**
 - FullItemRefundStrategy — returns the full transaction amount when `refundAll=true` and the transaction is within the 30-day return window from `createdAt`
 - PartialItemRefundStrategy — returns the sum of `(orderItem.priceAtPurchase * orderItem.quantity)` across only the `orderItemIds` listed in the request, when `refundAll=false`, the items belong to the transaction's order, and the transaction is within the 30-day return window
 - NoRefundStrategy — returns zero and a “return window expired” reason code when the transaction is older than 30 days from `createdAt`
- A `RefundStrategySelector` (not the service itself) chooses which strategy to instantiate based on `refundAll`, `orderItemIds`, and transaction age. The service only calls `selector.select(transaction, request).calculateRefund(transaction, request)`.

Test scenario:

- a) Via reflection, assert that the class `RefundStrategy` exists and is an interface with exactly one abstract method whose name is `calculateRefund`.

- b) Via reflection, assert that `FullItemRefundStrategy`, `PartialItemRefundStrategy`, and `NoRefundStrategy` all exist and implement `RefundStrategy`.
- c) Via reflection, assert that a class named `RefundStrategySelector` (or `RefundStrategyFactory`) exists with a `select`-like method returning `RefundStrategy`.
- d) Call S5-F12 with a transaction within the 30-day window and `refundAll=true` → verify the audit trail in MongoDB contains the strategy name `FullItemRefundStrategy` (or equivalent identifier stored with the event).
- e) Call S5-F12 with a transaction within the window, `refundAll=false`, and a non-empty `orderItemIds` list → audit trail records `PartialItemRefundStrategy`.
- f) Call S5-F12 with a transaction older than 30 days → returns 400, audit trail records `NoRefundStrategy`.
- g) Inspect the billing-service source: the refund service method must not contain `if (refundAll)` or equivalent branching.

3.3 Observer Pattern — Event Logging

Purpose: When entity state changes (user registers, order delivered, transaction refunded, etc.), events must be logged to MongoDB. Observer decouples logging from business logic.

Structure:

- `EntityObserver` interface with `onEvent(String eventType, Object payload)` method
- Concrete observer: `MongoEventLogger` that writes events to the appropriate MongoDB collection
- Subjects (services) maintain an observer list and call `notifyObservers(eventType, payload)` on state changes. Provide `register(observer)` / `unregister(observer)` methods on each subject.
- Each of the 5 services owns its own per-service `MongoEventLogger` instance; observer registration is not shared across services.

Failure policy Catching Exception for Mongo related features (Soft Dependency): The `MongoEventLogger` must catch any Mongo exception, log it at WARN level (`log.warn()`), and **not rethrow**. The upstream Postgres transaction must **not** be rolled back on a Mongo write failure.

Spring vs classical GoF: Spring provides a built-in Observer mechanism via `ApplicationEventPublisher` + `@EventListener`. For M2, **you must implement the classical GoF Observer** (explicit interface + register/notify methods) for learning purposes. No method in any of the 5 services annotated with `@EventListener` may write to MongoDB — all MongoDB event writes must flow through your classical Observer chain. You may, of course, still use Spring for everything else in your services.

Where applied:

- S1 register/login/role-change → observers log `AuthEvent` to `auth_events`
- S2 product updates → observers log to `product_events`
- S3 order state changes → observers log lifecycle events
- S4 shipment tracking → observers log to `shipment_events`
- S5 refund processed → observers log to `transaction_audit_trail`
- **Also applied to your M1 write endpoints** (see Section 4)

Test scenario:

- a) Via reflection, assert that `EntityObserver` exists as an interface with an `onEvent(String, Object)` method.
- b) Via reflection, assert `MongoEventLogger` class exists and implements `EntityObserver`.
- c) Via static analysis (class-file or source scan), assert that no method in the `User/Product/Order/Shipping/Billing` services annotated with `@EventListener` writes to MongoDB at all — all event-logging to MongoDB must flow through the classical GoF Observer chain.
- d) Register a fresh user via `POST /api/auth/register` → verify exactly one `REGISTERED` document exists in `auth_events` collection with matching `userId` and recent `timestamp`.
- e) Log in as that user → verify a `LOGGED_IN` document is added.
- f) Trigger an M1 write (e.g., `PUT /api/users/{id}/preferences` from S1-F2) → verify the corresponding event document is written to MongoDB (confirms the retrofit path).
- g) Unregister all observers from a service in a unit test and repeat the write → no MongoDB event should appear (proves the logging path goes through the observer chain, not a direct Mongo call).

3.4 Chain of Responsibility — JWT Filter Chain

Purpose: JWT authentication has multiple sequential steps (extract token, verify signature, load user, check role). Each step is a separate handler in a chain.

Structure:

- `AuthHandler` abstract class (or interface) with `handle(AuthContext ctx)` and `setNext(AuthHandler next)` methods, where `AuthContext` carries the HTTP request, extracted token, authenticated user, and required role
- Concrete handlers (in order): `TokenExtractionHandler` (401 if missing), `SignatureValidationHandler` (401 if invalid/expired), `UserLoaderHandler` (401 if user not found in PG), `RoleAuthorizationHandler` (403 if insufficient role for the endpoint)
- Each handler does its job and passes to the next; returns an error result immediately if any check fails

Spring Security integration: Spring Security's `SecurityFilterChain` is itself a chain of responsibility at the framework level. You **must not** replace Spring Security's filter chain. Instead, build your custom `AuthHandler` chain **inside** your `JwtAuthenticationFilter.doFilterInternal()` method:

- Spring Security's filter chain invokes your `JwtAuthenticationFilter` once per request
- Inside `doFilterInternal`, construct (or inject) the head of your custom `AuthHandler` chain and call `head.handle(new AuthContext(request))`
- If any handler in your chain fails, write the appropriate status code (401 or 403) to the response and short-circuit by **not** calling `filterChain.doFilter(...)`
- If all handlers succeed, populate the Spring Security context (`SecurityContextHolder.getContext().setAuthentication(...)`) and call `filterChain.doFilter(...)`

Test scenario:

- a) Via reflection, assert `AuthHandler` class/interface exists with `setNext(AuthHandler)` and `handle(...)` methods.

- b) Via reflection, assert at least 3 concrete subclasses of `AuthHandler` exist (e.g., `TokenExtractionHandler`, `SignatureValidationHandler`, `UserLoaderHandler`, and optionally `RoleAuthorizationHandler`).
- c) Call a protected M1 endpoint (e.g., `GET /api/users/1`) with `Authorization` header absent → returns 401 (`TokenExtractionHandler` failed).
- d) Call with `Authorization: Bearer invalid.token.here` (invalid signature) → returns 401 (`SignatureValidationHandler` failed).
- e) Delete the user from PG, then call with a valid token issued for that user → returns 401 (`UserLoaderHandler` failed).
- f) Call `PUT /api/users/1/role` with a `CUSTOMER` token → returns 403 (`RoleAuthorizationHandler` failed).
- g) Call the same endpoint with an `ADMIN` token → succeeds (chain passes through).
- h) Inspect `JwtAuthenticationFilter.doFilterInternal()` source: the grader parses it and confirms that the method body invokes the head of the `AuthHandler` chain rather than duplicating the extraction/validation/authorization logic inline.

3.5 Builder Pattern — Dashboard DTOs

Purpose: Dashboard and analytics DTOs have 5–10+ fields. Builder improves readability and allows optional fields.

Structure:

- Static inner `Builder` class on each dashboard DTO (or external `Builder` class)
- `builder()` static method to start construction
- Fluent setters returning `this`
- `build()` method returning the DTO instance

Builder with Java records: The project convention is “Java records for DTOs.” Records do not naturally support Builder (the canonical constructor takes all fields positionally). Two options, both accepted by the grader:

- **Convert to class:** Turn the record into a class with a static inner `Builder`.
- **External builder on the record:** Keep the record and write an external `<DtoName>Builder` class whose `build()` method constructs the record via its canonical constructor.

Where applied in M2: S2-F12 `ProductCatalogDashboardDTO`, S3-F10 `OrderAnalyticsDashboardDTO`, S4-F10 `ShippingAnalyticsDTO`, S5-F10 `CategoryRevenueDTO`.

Amazon M1 retrofit scope: Apply Builder to M1 DTO-returning features whose DTOs have 5+ fields. These DTOs for Amazon specifically are:

- **S1-F3, S1-F6, S1-F8, S1-F9:** Builder required (all return DTOs with 5+ fields)
- **S2-F3, S2-F6, S2-F9:** Builder required
- **S2-F8 (Verify Product Review): No Builder retrofit** — this feature returns an entity (the updated Product with its reviews), not a DTO
- **S3-F3, S3-F6, S3-F9:** Builder required
- **S3-F8 (Add Items to Existing Order): No Builder retrofit** — returns the updated Order entity with its items, not a DTO

- **S4-F3, S4-F6, S4-F8, S4-F9:** Builder required
- **S5-F3, S5-F6, S5-F8, S5-F9:** Builder required

Test scenario:

- a) Via reflection, iterate every DTO class returned by an M2 dashboard feature: assert each has an accessible static `builder()` method, that chaining setters returns the Builder, and that `build()` returns the correct DTO type.
- b) Via reflection, iterate the in-scope Amazon M1 DTOs (`UserOrderSummaryDTO`, `TopBuyerDTO`, `ProductSalesDTO`, `TopProductDTO`, `LowStockAlertDTO`, `OrderEstimateDTO`, `OrderAnalyticsDTO`, `OrderDetailsDTO`, `NearbyShipmentDTO`, `CarrierSummaryDTO`, `DelayedShipmentDTO`, `UserTransactionSummaryDTO`, `RevenueReportDTO`, `TransactionDetailsDTO`, `VoucherUsageDTO`): each must have a Builder.
- c) In an integration test, call S2-F12 GET `/api/products/catalog/dashboard` → verify the response is correctly populated (proves Builder is used to construct the DTO before serialization).
- d) Call M1 S1-F3 GET `/api/users/{id}/order-summary` → verify it still returns the expected fields (proves the retrofit did not break M1 behavior).
- e) Compile-time or static analysis check: confirm S2-F8 and S3-F8 do **not** use Builder (they return entities, not DTOs).

3.6 Singleton Pattern — `JwtConfigurationManager`

Purpose: A class holding shared, immutable JWT configuration (secret key, expiration, algorithm) should have exactly one instance.

Structure:

- Private constructor
- Static `getInstance()` method with thread-safe initialization (double-checked locking or eager init)
- Single private static field holds the instance

Loading config in a non-Spring class: Since `JwtConfigurationManager` is not a Spring bean, it cannot use `@Value` or `@ConfigurationProperties`. Use one of the following accepted approaches:

- **(Recommended) Environment variables with fallback defaults:** Inside the private constructor, read `System.getenv("JWT_SECRET")`, `System.getenv("JWT_EXPIRATION_MS")`, etc., with sensible fallback defaults for local dev. Docker Compose already sets the required env vars.
- **Singleton-bridge pattern:** A Spring `@Configuration` bean reads `application.yml` via `@Value`, then pushes the values into the singleton via a static setter (e.g., `JwtConfigurationManager.initConfig(secret, expirationMs)`) during `@PostConstruct`. The singleton then serves those values through its instance methods.

Both approaches are accepted by the grader as long as the class itself is not a Spring bean and `getInstance()` returns a consistent singleton reference.

Spring vs classical GoF (important): Spring's `@Component`/`@Service` beans are singletons by default, but they are managed by the Spring container, not by the GoF pattern. For M2 you must have **exactly one** class implemented as a classical GoF Singleton:

- **Implement `JwtConfigurationManager` as a classical Singleton** (private constructor + `getInstance()`). This class is not a Spring bean. Do not annotate it with `@Component`.

- `JwtService` remains a `Spring @Service`. It obtains JWT config via `JwtConfigurationManager.getInstance()` rather than via `@Autowired`.
- All other infrastructure classes (controllers, services, repositories) remain Spring-managed.

Test scenario:

- Via reflection, assert `JwtConfigurationManager` exists and has exactly one constructor declared with `private` access.
- Via reflection, assert `getInstance()` is a public static method returning `JwtConfigurationManager`.
- Call `getInstance()` twice in the test and assert `ref1 == ref2` (reference equality, not just `.equals()`).
- Spawn 10 parallel threads each calling `getInstance()` → all must return the same reference (thread-safety under contention).
- Via reflection, assert the class is NOT annotated with `@Component`, `@Service`, `@Configuration`, or any other Spring stereotype.
- In an integration test, verify that `JwtService` (a Spring bean) correctly reads the configured secret via `JwtConfigurationManager.getInstance()` — issue and validate a token and confirm the flow works.

3.7 Factory Pattern — Event Creation

Purpose: MongoDB events have different types across services (`AuthEvent`, `ProductEvent`, `OrderEvent`, `ShipmentEvent`, `TransactionAuditEvent`). A `EventFactory` creates the right concrete event type based on input.

Structure:

- A common `MongoEvent` interface that all 5 concrete event classes implement, with methods `getId()`, `getTimestamp()`, `getAction()`, `getDetails()`. (See Section 7.1 for the common interface definition.)
- `EventFactory` class with `createEvent(EventType type, Map<String, Object> params)` method
- The factory dispatches on `type` (an enum with values `AUTH`, `PRODUCT`, `ORDER`, `SHIPMENT`, `TRANSACTION_AUDIT`) and returns the appropriate concrete class, typed as `MongoEvent`

How Observer and Factory compose: On an M1 write completing, the service calls `notifyObservers(eventType, payload)`. The `MongoEventLogger` observer receives the call, asks the `EventFactory` for the right event subtype, and persists the event to MongoDB via Spring Data.

Test scenario:

- Via reflection, assert `MongoEvent` interface exists with methods `getId`, `getTimestamp`, `getAction`, `getDetails`.
- Via reflection, assert all 5 event classes (`AuthEvent`, `ProductEvent`, `OrderEvent`, `ShipmentEvent`, `TransactionAuditEvent`) implement `MongoEvent`.
- Via reflection, assert `EventFactory` exists with a `createEvent(EventType, Map<String, Object>)` method.
- In a unit test, call `EventFactory.createEvent(AUTH, params)` → assert the returned object is assignable to `AuthEvent` and its fields match the params.

- e) Repeat for each of the 5 `EventType` values.
- f) Call `EventFactory.createEvent(TRANSACTION_AUDIT, ...)` → the returned `TransactionAuditEvent` must expose `method` and `amount` fields (service-specific fields added on top of the common interface).
- g) Integration test: register a user via `POST /api/auth/register` → inspect the resulting MongoDB document in `auth_events`. Its type and fields must match what `EventFactory.createEvent(AUTH, ...)` would produce (proves services go through the factory).
- h) Source-scan check: verify no service class contains `new AuthEvent(...)`, `new ProductEvent(...)`, etc. — all event construction must go through the factory.

3.8 Adapter Pattern — NoSQL Result → DTO Conversion

Purpose: Results from different NoSQL databases have different raw shapes (MongoDB Document, Elasticsearch `SearchHit`, Neo4j `Record`, Cassandra `Row`). Adapters convert each raw source to the specific DTO the service returns.

Structure:

- Each adapter has a single `adapt(source) → targetDto` method. There is **no** universal “Entity-Dto” base type — each adapter converts to its specific domain DTO.
- One adapter per NoSQL source used in the service:
 - S1: `MongoDocumentAdapter` (for `AuthEvent` documents)
 - S2: `MongoDocumentAdapter`, `ElasticsearchHitAdapter`
 - S3: `MongoDocumentAdapter`, `Neo4jRecordAdapter`
 - S4: `MongoDocumentAdapter`, `CassandraRowAdapter`
 - S5: `MongoDocumentAdapter`

M1 retrofit scope (conditional): In M1, some features that return DTOs from native SQL used `Object[]` row projections while others used JPQL constructor expressions or `@Query` DTO projections. The Adapter retrofit is **conditional**:

- **If** your M1 feature uses `Object[]` (S1-F3 in the M1 spec explicitly mandates this) → wrap the mapping in an `ObjectArrayDtoAdapter` class rather than doing inline mapping in the service.
- **If** your M1 feature uses JPQL constructor expressions or DTO projections → no Adapter needed; the existing code satisfies the pattern by construction.

Test scenario:

- a) Via reflection, assert that for each NoSQL source used by each service, a corresponding adapter class exists: `MongoDocumentAdapter` in all services; `ElasticsearchHitAdapter` in product-service; `Neo4jRecordAdapter` in order-service; `CassandraRowAdapter` in shipping-service.
- b) Via reflection, assert each adapter has an `adapt(...)` method whose return type matches the service’s domain DTO.
- c) In a unit test, pass a mock MongoDB Document to `MongoDocumentAdapter.adapt(...)` → verify the returned DTO has fields populated from the document’s keys.
- d) In a unit test for product-service: pass an `Elasticsearch SearchHit` to `ElasticsearchHitAdapter.adapt(...)` → verify the returned product DTO.
- e) For S1-F3 (the only M1 feature that explicitly mandates `Object[]`): via reflection or source scan, verify an `ObjectArrayDtoAdapter` (or similar) class exists and is used to convert the native SQL `Object[]` rows into `UserOrderSummaryDTO`. Invoke S1-F3 in integration and verify the output is correct.

- f) For any other in-scope M1 F3/F6/F9 feature that the team chose to implement with `Object[]`: check the corresponding adapter exists. Features implemented via JPQL constructor expressions or DTO projections are **exempt** from this check.

3.9 Grading Summary

Each pattern contributes roughly equally to the design patterns portion of the M2 grade. The grader performs:

- **Static analysis** — reflective inspection of class structure (interfaces, methods, constructors)
- **Behavioral verification** — integration tests that exercise the patterns indirectly through M2 features

4 M1 Modifications Required

Before you can build M2 features, you must **modify your existing M1 codebase** in specific ways. These modifications are prerequisites — M2 features and cross-cutting requirements depend on them.

4.1 User Entity — Password Hashing

Your M1 User entity already has a `password` field. In M1 it was stored as plaintext (or loosely). In M2:

- All new user registrations must hash passwords with **BCrypt** before saving
- Existing M1 test users/seeds should be re-created with hashed passwords (or hash them on first login)
- The `password` field must **never** be returned in any API response (including existing M1 `GET /api/users/{id}` endpoint — filter it out in the DTO)

Test scenario:

- a) Register a user via `POST /api/auth/register` with password `"securePassword123"`.
- b) Query the `users` table directly and assert the stored `password` column starts with `"$2a$"`, `"$2b$"`, or `"$2y$"` (BCrypt hash prefix) and its length is 60 characters.
- c) Assert the stored password is **not** equal to `"securePassword123"`.
- d) `POST /api/auth/login` with the same credentials → succeeds (BCrypt verify works).
- e) Call `GET /api/users/{id}` (M1 endpoint) with a valid token → inspect the JSON response body: the `password` field must be absent or explicitly `null`. Same for any other endpoint that returns a User (e.g., S1-F1 search, S1-F3 summary, S1-F8 profile).
- f) Re-seed the database using the seed mechanism → assert every seeded user's `password` column is a BCrypt hash, not plaintext.

4.2 User Entity — Role Values (Additive)

Your M1 User entity already has a `role` ENUM with values `CUSTOMER` and `ADMIN`. **Do not remove or rename existing M1 role values.** M2 modifies this only additively:

- **Keep** `CUSTOMER` and `ADMIN` as they are

- **Default role on registration:** `CUSTOMER` (Amazon's M1 default). `ADMIN` is never assigned at registration.
- **ADMIN assignment:** Only via the role management endpoint (`PUT /api/users/{id}/role`) which itself requires `ADMIN`. Seed at least one `ADMIN` user via your M1 seed mechanism so role management can be tested.

JWT role claim: The token carries the user's role. Authorization logic:

- **USER-level endpoints:** Accept any authenticated user (`CUSTOMER` or `ADMIN`) — this is the default for M2 features.
- **ADMIN-only endpoints:** Require `role == ADMIN`. Only the role management endpoint uses this.

Test scenario:

- Query PostgreSQL metadata (`information_schema.columns` or the Postgres enum `pg_enum`) to confirm the `role` ENUM type contains both `CUSTOMER` and `ADMIN` values.
- Register a fresh user via `POST /api/auth/register` with valid data → query the `users` table and assert the new row's `role` column is `CUSTOMER`.
- Decode the returned JWT token → assert the `role` claim is `CUSTOMER`.
- Log in as a seeded `ADMIN` user → decode token and assert `role` claim is `ADMIN`.
- Attempt to register with a request body containing `"role": "ADMIN"` (a bad-faith attempt) → the server must ignore that field and still create a `CUSTOMER`.
- With the `CUSTOMER` token, call `PUT /api/users/{id}/role` → 403 Forbidden.
- With the `ADMIN` token, call `PUT /api/users/{id}/role` with body `{"role": "ADMIN"}` → 200, the target user's role changes.
- With the `CUSTOMER` token, call any regular M2 feature (e.g., S2-F10 full-text search) → succeeds (USER-level endpoints accept both `CUSTOMER` and `ADMIN`).

4.3 Existing M1 Endpoints — JWT Authentication

All existing M1 endpoints (45 feature endpoints + CRUD endpoints on all entities) must now require a valid JWT token, except:

- `POST /api/auth/register` (new in M2)
- `POST /api/auth/login` (new in M2)
- Health check endpoints

Test scenario:

- Call `POST /api/auth/register` (public) → succeeds without any `Authorization` header.
- Call `POST /api/auth/login` (public) → succeeds without a token.
- Call any M1 feature endpoint (e.g., `GET /api/products/search`, M1 S2-F1) without token → returns 401.
- Call the same endpoint with `Authorization: Bearer invalid` → returns 401.
- Call the same endpoint with a valid token → succeeds with 200 and the expected M1 response shape.

- f) Call every CRUD endpoint (GET /api/users, GET /api/users/{id}, POST /api/users, PUT /api/users/{id}, DELETE /api/users/{id}, and the equivalents on all 10 Amazon entities) without token → each returns 401. With a valid token → each returns the expected M1 status code.
- g) Health check endpoints (e.g., GET /api/users/health) return 200 without a token.
- h) Confirm by grepping the test results that all 45 M1 feature tests + CRUD tests pass under M2 auth.

4.4 Existing M1 Read Endpoints — Redis Caching (Explicit Enumeration)

All M1 read-heavy endpoints must be cached in Redis. Below is the complete enumeration — nothing is left open.

4.4.1 M1 Feature GET Endpoints That Must Be Cached (Amazon: 27 total)

The canonical M1 pattern is “F2, F4, F7 are writes, the other 6 are reads,” but **Amazon deviates at three specific features** that are writes despite their F-number:

Service	Feature	Verb	Why it is a write
S2 (Product)	S2-F8 Verify Product Review	PUT	Modifies <code>ProductReview.verified</code> and JSONB meta
S3 (Order)	S3-F8 Add Items to Existing Order	POST	Creates new <code>OrderItem</code> rows
S5 (Billing)	S5-F5 Apply Voucher to Transaction	POST	Creates <code>TransactionVoucher</code> join row and mutates V

These are **excluded** from the cached-reads list and **included** in the invalidating-writes list (§4.4.4). The explicit cached-read enumeration for Amazon is:

Service	Cached GET feature IDs
S1 (User)	F1, F3, F5, F6, F8, F9
S2 (Product)	F1, F3, F5, F6, F9 (F8 is a write)
S3 (Order)	F1, F3, F5, F6, F9 (F8 is a write; F3 is a POST estimate—see note)
S4 (Shipping)	F1, F3, F5, F6, F8, F9
S5 (Billing)	F1, F3, F6, F8, F9 (F5 is a write)
Total cached feature endpoints (Amazon)	

Note on S3-F3 (Get Order Price Estimate): In the M1 spec S3-F3 is a POST /api/orders/estimate that computes a price estimate without persisting. It is semantically read-only despite being POST. Cache it by request-body hash (TTL 5 min).

TTLs by feature type: F1 = 5 min (search), F3 = 10 min (DTO), F5 = 5 min (JSONB query), F6 = 10 min (report), F8 = 15 min (relationship DTO), F9 = 10 min (combined).

4.4.2 CRUD Baseline Endpoints That Must Be Cached

For every entity, **only** the get-by-ID endpoint is cached. List endpoints are **not** cached — they hit PostgreSQL on every request.

Endpoint	Cached?	TTL
GET /api/<entity> (list all)	No	—
GET /api/<entity>/<id> (get by ID)	Yes	15 min

Amazon entities (10): user, shipping-address, product, product-review, order, order-item, shipment, transaction, voucher, transaction-voucher. **10 GET-by-ID endpoints must be cached.**

4.4.3 Total

27 M1 feature endpoints + 10 M1 CRUD GET-by-ID endpoints = 37 M1 endpoints must be cached for Amazon. Plus the M2 feature GET endpoints.

4.4.4 Endpoints That Must Invalidate Caches (Writes)

Writes invalidate the detail cache of the specifically affected entity ID, plus feature-result caches whose output includes that entity. Since list endpoints are not cached, there is no list cache to invalidate.

Amazon M1 feature writes (18 total):

- S1 writes: F2, F4, F7 (3)
- S2 writes: F2, F4, F7, **F8 (Verify Product Review)** (4)
- S3 writes: F2, F4, F7, **F8 (Add Items to Existing Order)** (4)
- S4 writes: F2, F4, F7 (3)
- S5 writes: F2, F4, **F5 (Apply Voucher to Transaction)**, F7 (4)

M1 CRUD writes (30): POST, PUT, DELETE on each of the 10 Amazon entities.

- POST `/api/<entity>` (create) — nothing to invalidate (new entity has no cached detail yet)
- PUT `/api/<entity>/<id>` (update) — invalidate detail cache for that specific ID + any feature caches referencing that entity
- DELETE `/api/<entity>/<id>` — invalidate detail cache for that specific ID + any feature caches referencing that entity

Total Amazon M1 endpoints that invalidate: 18 feature writes + 30 CRUD writes = 48.

M2 writes that also invalidate M2 feature caches: In addition to the M1 writes above, the following M2 operations must invalidate feature-result caches so that read-side views see the change before the TTL expires:

- CC-2 Role management (PUT `/api/users/<id>/role`) — must invalidate `user-service::user::<id>` (detail) and all keys matching `user-service::S1-F12::*` for that `userId` (so the `ROLE_CHANGED` event appears in the activity feed immediately).
- Any M1 write that creates/updates an `Order`, `OrderItem`, or `Transaction` referencing a specific `productId` — must invalidate `product-service::S2-F12::*` so the Product Catalog Dashboard reflects the new activity.
- Any M1 write that creates/updates an `Order` — must invalidate `order-service::S3-F10::*` (the Order Analytics dashboard aggregates over all orders).
- Any M1 write that creates/updates a `Shipment` — must invalidate `shipping-service::S4-F10::*`.
- Any M1 write that creates/updates a `Transaction` (including S5-F4 Process Transaction, S5-F2 Refund, M2 S5-F12 Partial Item Refund) — must invalidate `billing-service::S5-F10::*` and `billing-service::S5-F11::*`.

NoSQL-writer → cached-reader invalidation (required): Several M2 features write to a NoSQL store as their primary action (no PG row mutates), but downstream M2 features cache reads of that same NoSQL store. The PG-write rules above do **not** cover these paths, so add these explicitly:

- **S4-F11** (POST /api/shipments/{id}/tracking) writes a new tracking row to Cassandra and emits a `TRACKING_RECORDED` event to MongoDB `shipment_events` — must invalidate `shipping-service::S4-F12::{shipmentId}` (the cached tracking timeline for that shipment, 5 min TTL) so S4-F12 reflects the new tracking point immediately. Must also invalidate `shipping-service::S4-F10::*` (the shipping analytics dashboard may aggregate over tracking-event counts).
- **S3-F11** (POST /api/orders/{orderId}/record-co-purchase) writes or updates `BOUGHT_TOGETHER` relationships (or increments existing ones) in Neo4j — must invalidate `order-service::S3-F12::*` (the cached recommendation results for all products, 5 min TTL) because the co-purchase graph changed. Use wildcard deletion rather than narrow targeting: a new edge between products P_1 and P_2 can alter recommendations for any product that shares an edge with P_1 or P_2 .
- **S2-F11** (POST /api/products/{id}/index) **and** every Product CRUD write that triggers the auto-indexing retrofit (see §4.5 “Additional M1 behavior retrofits”) — must invalidate `product-service::S2-F10::*` (the cached full-text search results, 5 min TTL). Product name / description / status / price changes alter ES relevance ranking, so stale cache can return deleted or renamed products.
- **S5-F10 / S5-F11 analytics writes** — whenever any Observer writes a **data-mutating** event to `transaction_audit_trail` (specifically `CREATED`, `COMPLETED`, `FAILED`, `REFUNDED`, `REFUND_DENIED`, `VOUCHER_APPLIED`), the billing-service analytics caches `billing-service::S5-F10::*` and `billing-service::S5-F11::*` must be invalidated. This covers both the normal-processing path (S5-F4, S5-F2, S5-F5, S5-F12 success branch) and the strategy-denial path (S5-F12 NoRefund-Strategy branch logs `REFUND_DENIED` without any PG mutation — the PG-write rules above miss this case, so the analytics caches would otherwise serve stale data until TTL expiry). Similarly, any Observer write of a data-mutating action to `product_events`, `order_events`, or `shipment_events` invalidates the corresponding S2-F12 / S3-F10 / S4-F10 analytics keys.
- **Pure observability actions do NOT invalidate caches** — specifically `ANALYTICS_VIEWED` and `DASHBOARD_VIEWED`. These actions are written by the four dashboard endpoints (S2-F12, S3-F10, S4-F10, S5-F10) on *every* invocation (including cache hits; see §10) purely for audit-trail / usage-telemetry purposes. They do not change what the analytics queries return. Triggering wildcard invalidation on them would create a self-defeating cycle: every cached dashboard call would log an observability event, which would wildcard-invalidate its own key, guaranteeing that the next call is always a miss and the cache never serves a hit. Exclude both `ANALYTICS_VIEWED` and `DASHBOARD_VIEWED` from the invalidation trigger in the Observer (match on the `action` field before invalidating).

These M2 invalidation rules compose with the M1 rules above; implement both.

4.4.5 Cache Key Convention

- Entity detail: `<service>::<entity>::<id>` (e.g., `user-service::user::42`)
- Feature result: `<service>::<featureId>::<param-hash>` (e.g., `user-service::S1-F3::42`)

4.4.6 Cache Invalidation Strategy (Wildcard Deletion)

Computing the exact set of feature-result caches affected by an entity change is infeasible without cache tags or reverse indexes. Use **wildcard key deletion** instead:

- When entity X with id N changes, delete the key `<service>::X::N` (entity detail).
- Also delete all feature-result keys matching the pattern `<service>::S{n}-F{m}::*` for every feature `S{n}-F{m}` whose output *may* include entity X (conservatively identified at design time).

- This over-invalidates (clears some entries that did not actually change). That is acceptable for M2 — correctness beats cache-hit optimality.

Use the Redis **SCAN + DEL** combination or **KEYS-UNLINK** (for small caches) to implement wildcard deletion.

Test scenario (entire §4.4 caching retrofit):

- For each of the 27 cached M1 feature GET endpoints listed in §4.4.1: call it twice with identical parameters and a valid token. Inspect Redis between calls (e.g., via `redis-cli KEYS 'user-service:*'`) and assert a matching key exists. Measure the second call's latency — it must be less than the first (cache hit).
- For each of the 10 CRUD GET `/api/<entity>/<id>` endpoints: call twice and verify a key `<service>::<entity>::<id>` exists in Redis after the first call.
- Call GET `/api/products` (list) twice → assert **no** cache key was created for the list (list endpoints are NOT cached).
- Fetch product ID 5 (cache it), then PUT `/api/products/5` with an update → verify the key `product-service::product::5` is removed from Redis. Next GET recomputes from PG.
- Fetch the S1-F3 DTO for user 10 (cache it), then update user 10 via PUT `/api/users/10/preferences` (S1-F2) → verify every key matching `user-service::S1-F3::*` is removed (wildcard invalidation).
- Delete product 7 via DELETE `/api/products/7` → the cached `product-service::product::7` must disappear.
- Disable Redis (e.g., stop the container) and retry a cached endpoint → it still returns correct data from PG (soft-dependency graceful degradation).
- Verify TTLs: fetch an S1-F1 search result (5 min TTL), wait 5m+, fetch again → cache was auto-evicted and the call recomputes.

4.5 Design Pattern Retrofits to M1

The 7 patterns from Section 3 apply to M2 code **and** to M1 retrofits as follows:

Pattern	Applies to M1?	Which M1 Amazon features	Notes
Strategy	No	—	New M2 code only (S5-F12 Partial Item Refund) — do NOT force onto any M1 method
Observer	Yes (universal)	All M1 write endpoints F2, F4, F7 per service + S2-F8, S3-F8, S5-F5 (Amazon-specific writes) + all CRUD writes	State change → <code>notifyObservers</code> → MongoDB event log
Chain of Responsibility	Yes (universal)	All M1 endpoints via the JWT filter chain	Inside <code>JwtAuthenticationFilter</code> (see Section 3.4)
Builder	Yes	S1-F3, S1-F6, S1-F8, S1-F9; S2-F3, S2-F6, S2-F9; S3-F3, S3-F6, S3-F9; S4-F3, S4-F6, S4-F8, S4-F9; S5-F3, S5-F6, S5-F8, S5-F9 (DTO-returning features with 5+ fields)	Excludes S2-F8 and S3-F8 for Amazon — they return entities, not DTOs
Singleton	No	—	New M2 code only (<code>JwtConfigurationManager</code>)
Factory	Yes (indirect)	M1 write endpoints trigger Observer; Observer calls <code>EventFactory</code>	<code>EventFactory</code> creates the right <code>MongoEvent</code> subtype
Adapter	Conditional	Any M1 feature that uses <code>Object[]</code> native SQL projection (S1-F3 mandates <code>Object[]</code> ; other F3/F6/F9 only if you chose <code>Object[]</code> in M1)	If you used JPQL constructor expressions or DTO projections, no Adapter retrofit is required

Important: Strategy Pattern is used **only** in M2 code (S5-F12 Partial Item Refund). Do not force it onto any M1 method.

Composition workflow (Observer + Factory + Adapter):

- A write endpoint (e.g., M1 S1-F2 Update User Preferences) completes its PG update.
- The service calls `notifyObservers("USER_UPDATED", userPayload)` before returning. The first argument is the **action string** (what happened); it is not the `EventType` passed to the factory.
- `MongoEventLogger` observer receives the call. Each of the 5 services instantiates its own `MongoEventLogger` bound to a **fixed** `EventType` at construction time (user-service binds `AUTH`, product-service binds `PRODUCT`, order-service binds `ORDER`, shipping-service binds `SHIPMENT`, billing-service binds `TRANSACTION_AUDIT`). The logger does **not** infer `EventType` from the action string.
- The logger builds the factory input: copy the action string into `params.put("action", eventType)` plus the rest of the payload, then calls `EventFactory.createEvent(boundEventType, params)`.

- e) The factory returns the matching concrete `MongoEvent` subtype (e.g., `AuthEvent`) typed through the common interface.
- f) The logger persists the event to MongoDB via Spring Data's repository.

Adapter sits separately on **read paths** (converting NoSQL query results or M1 `Object[]` results to DTOs) — it does not participate in the write-side composition above.

Billing Service transaction_audit_trail population (important): The `transaction_audit_trail` collection powers M2 S5-F11 (GET `/api/transactions/{transactionId}/lifecycle`). Without writes to this collection, S5-F11 returns empty results. Therefore:

- **M1 S5-F4** (Process Transaction for Order) must write two events to `transaction_audit_trail`: a `CREATED` event when the Transaction row is first populated/updated from `PENDING`, and a `COMPLETED` event when the transaction status transitions to `COMPLETED`.
- **M1 S5-F2** (Process Refund — the original M1 simple refund) must write a `REFUNDED` event.
- **M2 S5-F12** (Partial Item Refund) must also write a `REFUNDED` event with richer details (strategy name, refunded item IDs, refund amount) on the success path, and a `REFUND_DENIED` event on the `NoRefundStrategy` denial path.

These are observer-driven writes — use the Observer retrofit (row 2 of the table above) rather than inline Mongo calls in the service methods.

Additional M1 behavior retrofits (also required for grading — not dedicated subsections to preserve §4's cross-ref numbering):

- **S5-F4 gateway-failure simulation (query param):** M1's POST `/api/transactions/order/{orderId}` must accept an optional `?simulateFailure=true` query parameter. When set, S5-F4 short-circuits to `Transaction.status = FAILED` in PostgreSQL and writes a `FAILED` event to `transaction_audit_trail` (via Observer) instead of the normal `CREATED`→`COMPLETED` flow. When absent or `false`, S5-F4 behaves identically to its M1 spec. This affordance exists solely so the S5-F11 test scenario (and any Strategy denial-path test) can produce failed-transaction data; it is not a production flow. See the `FAILED` row of §7.1.6 `TransactionAuditEvent` for the full write-conditions table.
- **Product CRUD auto-index to Elasticsearch:** M1's Product CRUD controller must auto-sync to the M2 Elasticsearch index on every write. POST `/api/products` and PUT `/api/products/{id}` must **re-index** the product's search document (as if S2-F11 had been called explicitly); DELETE `/api/products/{id}` must **remove** the document from the index. See §10 S2-F11 step e for the exact expected behavior. Implement via a `@PostPersist/@PostUpdate/@PostRemove` JPA entity listener or a service-level hook; do **not** inline the ES call into every CRUD controller method.

Test scenario (design pattern retrofits):

- a) Call M1 S1-F2 PUT `/api/users/{id}/preferences` → verify an event document appears in `auth_events` (Observer retrofit fired on the write).
- b) Call M1 S2-F8 (Verify Product Review) → event in `product_events`.
- c) Call M1 S3-F8 (Add Items to Existing Order) → event in `order_events`.
- d) Call M1 S5-F5 (Apply Voucher to Transaction) → event in `transaction_audit_trail`.
- e) Call M1 S5-F4 (Process Transaction) → `CREATED` and `COMPLETED` events appear in `transaction_audit_trail`.
- f) Call M1 S5-F2 (Process Refund) → `REFUNDED` event appears.

- g) Call a CRUD write (e.g., `POST /api/products`) → event in `product_events`, and the product must appear in the Elasticsearch index (auto-index retrofit).
- h) Call M1 S1-F3 `GET /api/users/{id}/order-summary` → response is well-formed; via source scan, confirm the DTO is constructed via its Builder (not via `new UserOrderSummaryDTO(...)` directly inside the service).
- i) Call M1 S1-F3 and confirm that the `Object[]` result from the native SQL query is converted to `UserOrderSummaryDTO` via an Adapter class (not inline mapping in the service).
- j) Verify that Strategy is NOT used anywhere in M1 service classes (grep for `RefundStrategy`, `Strategy` outside the billing service's S5-F12 package).

4.6 Transaction.transactionDetails — Additive Refund Keys

M2 S5-F12 (Partial Item Refund) records richer refund data than M1's simple S5-F2 refund. M1's `Transaction.transactionDetails` JSONB schema is `{gatewayResponse, cardLastFour, receiptUrl, failureReason}`, and M1 S5-F2 adds `refundReason` and `refundedAt` when it runs. M2 S5-F12 **additively extends** this JSONB with further keys:

- **refundAmount** — numeric amount (currency units) returned to the customer by this refund operation.
- **refundedItems** — a list of objects, each with `{orderId, quantity, amount}`, describing which items were refunded and by how much. For `FullItemRefundStrategy`, this lists every item in the transaction's order; for `PartialItemRefundStrategy`, it lists only the items requested in the refund body.
- **refundStrategy** — the class name of the `RefundStrategy` subtype that produced the refund (e.g., `"FullItemRefundStrategy"`, `"PartialItemRefundStrategy"`, `"NoRefundStrategy"`).
- **refundedAt** — ISO-8601 timestamp of the refund operation (already written by M1 S5-F2; M2 S5-F12 follows the same convention).

No M1 code change required: Unlike a JSONB extension that also requires an M1-side retrofit (such as Talabat's `deliveryFee`), these keys are written only by the new M2 S5-F12 endpoint. M1 code is not affected. Pre-M2 Transactions that were processed before M2 went live simply do not have these keys in their `transactionDetails` — readers of the JSONB must treat missing keys as absent (do not throw).

4.7 Summary Checklist

Before starting M2 features, ensure your M1 codebase:

- ☐ Password hashing (BCrypt) applied to registration and seed data
- ☐ ADMIN role present; CUSTOMER preserved as default
- ☐ At least one ADMIN user seeded
- ☐ All endpoints (except register/login/health) require JWT
- ☐ 27 M1 feature GET endpoints + 10 CRUD GET-by-ID endpoints cached (37 total — see Section 4.4)
- ☐ M1 write endpoints invalidate the correct detail/feature caches
- ☐ M1 write endpoints notify observers (event logging)
- ☐ M1 complex DTOs (F3/F6/F8/F9) use Builder pattern (excluding S2-F8 and S3-F8 which return entities)
- ☐ M1 `Object[]` native SQL results (F3/F6/F9) use Adapter pattern

- ❑ M1 S5-F4 accepts optional `?simulateFailure=true` query param that writes a **FAILED** audit event — see §4.5
- ❑ M1 Product CRUD auto-indexes to Elasticsearch on every POST/PUT/DELETE — see §4.5
- ❑ M2 S5-F12 writes the additive refund keys (`refundAmount`, `refundedItems`, `refundStrategy`) into `Transaction.transactionDetails` — see Section 4.6
- ❑ All existing M1 tests still pass

5 Authentication & Authorization

5.1 Overview

The User Service (S1) becomes the authentication provider. It exposes register and login endpoints that return JWT tokens. All other services validate tokens independently using a shared secret key — no inter-service calls are needed for authentication.

5.2 JWT Token

- **Algorithm:** HMAC-SHA256
- **Secret:** A shared key configured in each service's `application.yml` (e.g., `jwt.secret`). The shared secret must be at least **32 bytes (256 bits)** of entropy when decoded — JJWT's HS256 signer enforces this minimum and throws `WeakKeyException` at construction time on shorter keys. Use `Keys.secretKeyFor(SignatureAlgorithm.HS256)` to generate a compliant key, or configure any 44-character Base64-encoded string of random bytes (44 Base64 chars decode to 32 bytes). Short readable strings like "mySecret123" will fail at the first token-issue attempt.
- **Payload:** The token contains the user's email (as the `sub` subject claim), the user's numeric `User.id` (as a custom `uid` claim), the user's role (as a `role` claim), issued-at (`iat`), and expiration (`exp`). The ownership check in §10 (S1-F12 activity feed) compares the `uid` claim directly against the path `{id}` — no additional PG lookup is required on the hot path.
- **Header format:** `Authorization: Bearer <token>`
- **Expiration:** Configurable (e.g., 24 hours)

5.3 Roles (Additive — Preserve M1 Values)

The M1 User entity already has a `role` ENUM with values `CUSTOMER` and `ADMIN`. M2 preserves these additively — do not rename or remove existing M1 role values.

Role	Permissions
CUSTOMER (M1 default)	Access all standard endpoints, manage own data
ADMIN (universal)	Everything CUSTOMER can do, plus: change any user's role, verify any review

- **Default role on registration:** `CUSTOMER` (Amazon's M1 default)
- `ADMIN` is never assigned on register
- Only an `ADMIN` can change another user's role via `PUT /api/users/{id}/role`
- Only an `ADMIN` can verify a product review via M1 S2-F8 (already enforced in M1)
- The JWT token's `role` claim contains the user's actual role value
- Seed at least one `ADMIN` user so role management and M1 S2-F8 can be tested

5.4 Security Configuration (Per Service)

Each of the 5 services must have:

- A security configuration with a JWT filter that intercepts every request
- Stateless session management (no server-side sessions)
- CSRF disabled (since this is a token-based API)
- Public endpoints: `/api/auth/register` and `/api/auth/login` (on User Service only), and health checks
- All other endpoints require a valid JWT token

5.5 Password Handling

The User entity already has a `password` field from M1. In M2, you must ensure:

- Passwords are hashed with BCrypt before being stored
- The plaintext password is **never** stored or returned in API responses
- Login verifies the provided password against the stored hash

6 Database Architecture

6.1 Six Databases (Versions Match Task 2 Exactly)

All teams use the same 6 databases. **Docker image versions match Task 2** so students who completed Task 2 can reuse their stack without reconfiguration.

Database	Port	Docker Image	Architectural Role
PostgreSQL	5432	<code>postgres:17</code>	Primary relational data (from M1). PG 17 mandatory — PG 18 breaks Hibernate
MongoDB	27017	<code>mongo:latest</code>	Activity logs, event history, analytics — used by all services
Redis	6379	<code>redis:latest</code>	Caching layer for read-heavy endpoints — used by all services
Elasticsearch	9200	<code>elasticsearch:8.19.12</code>	Full-text search across products (S2). Pinned version.
Neo4j	7687	<code>neo4j:latest</code>	Co-purchase recommendation graph (S3)
Cassandra	9042	<code>cassandra:latest</code>	Time-series shipment tracking events (S4)

6.2 Memory Limitations (Required)

Running 6 databases + 5 Spring Boot services strains laptop RAM. Apply these memory limits to avoid system freezes:

Service	Memory Limit	Rationale
Redis	256 MB max, <code>allkeys-lru</code> eviction	Bounded cache
Elasticsearch	512 MB JVM heap (<code>ES_JAVA_OPTS=-Xms512m -Xmx512m</code>)	Default heap is 4 GB
Cassandra	512 MB heap (<code>MAX_HEAP_SIZE=512M, HEAP_NEWSIZE=128M</code>)	Default is 1–2 GB
Neo4j	512 MB heap (<code>NEO4J_server_memory_heap_max_size=512m</code>)	Default can exceed 1 GB
PostgreSQL, MongoDB	Default	Well-behaved at defaults

Estimated total memory for the stack: PostgreSQL (~250 MB) + MongoDB (~400 MB) + Redis (256 MB capped) + Cassandra (~700 MB) + Neo4j (~600 MB) + Elasticsearch (~700 MB) + 5 Spring Boot apps ($5 \times 300 \text{ MB} = 1.5 \text{ GB}$) = **~4.5 GB** for the stack alone.

6.3 Dependency Model

Following the pattern from Lab 5 and Task 2:

- **PostgreSQL:** Hard dependency — the service will not start without it
- **MongoDB, Redis, Elasticsearch, Neo4j, Cassandra:** Soft dependencies — if any of these are unavailable, the features that depend on them should degrade gracefully (try-catch), but the service should still start and serve PostgreSQL-based endpoints

6.4 Docker Compose Additions

Add these containers to your existing `docker-compose.yaml` alongside PostgreSQL and the 5 application services:

```
mongo:
  image: mongo:latest
  container_name: amazon-mongo
  ports:
    - "27017:27017"
  environment:
    MONGO_INITDB_ROOT_USERNAME: root
    MONGO_INITDB_ROOT_PASSWORD: rootpass
    MONGO_INITDB_DATABASE: amazonmongo
  volumes:
    - mongo-data:/data/db
  healthcheck:
    test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
    interval: 10s
    timeout: 5s
    retries: 5

redis:
  image: redis:latest
  container_name: amazon-redis
  ports:
    - "6379:6379"
  command: redis-server --requirepass redispass --maxmemory 256mb --maxmemory-policy allkeys-lru
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "redispass", "ping"]
    interval: 5s
    timeout: 5s
    retries: 5

elasticsearch:
  image: elasticsearch:8.19.12
  container_name: amazon-elasticsearch
  ports:
    - "9200:9200"
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
  volumes:
    - es-data:/usr/share/elasticsearch/data
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:9200/_cluster/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 30s
```

```

neo4j:
  image: neo4j:latest
  container_name: amazon-neo4j
  ports:
    - "7474:7474"
    - "7687:7687"
  environment:
    NEO4J_AUTH: neo4j/neo4jpass
    NEO4J_server_memory_heap_max__size: 512m
    NEO4J_server_memory_heap_initial__size: 256m
  volumes:
    - neo4j-data:/data
  healthcheck:
    test: ["CMD", "neo4j", "status"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 30s

cassandra:
  image: cassandra:latest
  container_name: amazon-cassandra
  ports:
    - "9042:9042"
  environment:
    CASSANDRA_CLUSTER_NAME: amazoncluster
    CASSANDRA_DC: datacenter1
    CASSANDRA_KEYSPACE: amazonks
    MAX_HEAP_SIZE: 512M
    HEAP_NEWSIZE: 128M
  volumes:
    - cassandra-data:/var/lib/cassandra
  healthcheck:
    test: ["CMD-SHELL", "cqlsh -e 'DESCRIBE KEYSPACES'"]
    interval: 15s
    timeout: 10s
    retries: 10
    start_period: 60s

volumes:
  mongo-data:
  es-data:
  neo4j-data:
  cassandra-data:

```

6.5 Reference application.yml Fragments

Each of your 5 services needs the connection properties for every database it uses. Note: M1 used `application.properties`. In M2 you should migrate to `application.yml` (YAML format is what the auto-grader expects).

Shared across all services

```

spring:
  datasource:
    url: jdbc:postgresql://postgres:5432/amazondb
    username: postgres

```

```
password: postgres
jpa:
  hibernate:
    ddl-auto: update
    show-sql: true
data:
  redis:
    host: redis
    port: 6379
    password: redispass

jwt:
  secret: "<Base64-encoded shared secret across all 5 services>"
  expiration: 86400000 # 24 hours in ms
```

All services — MongoDB (event logging)

```
spring:
  data:
    mongodb:
      uri: mongodb://root:rootpass@mongo:27017/amazonmongo?authSource=admin
```

Product service only — Elasticsearch

```
spring:
  elasticsearch:
    uris: http://elasticsearch:9200
```

Order service only — Neo4j

```
spring:
  data:
    neo4j:
      uri: bolt://neo4j:7687
      username: neo4j
      password: neo4jpass
```

Shipping service only — Cassandra

```
spring:
  cassandra:
    contact-points: cassandra
    port: 9042
    local-datacenter: datacenter1
    keyspace-name: amazonks
    schema-action: CREATE_IF_NOT_EXISTS
```

7 New Entity Models

Your existing PostgreSQL entities from M1 remain unchanged except for the password handling described in Section 4.1 and Section 5.5 and the additive refund keys on `Transaction.transactionDetails` described in Section 4.6. This section defines the **new** entities for the NoSQL databases.

7.1 MongoDB Document Entities (All Services)

Each service defines one MongoDB document class for its activity/event log. MongoDB entities must be **classes** (not records).

7.1.1 Common MongoEvent Interface

All 5 MongoDB event classes below implement a shared interface `MongoEvent` so that the `EventFactory` (Section 3.7) can return any concrete event typed as `MongoEvent`:

Method	Return type	Purpose
<code>getId()</code>	<code>String</code>	MongoDB ObjectId
<code>getTimestamp()</code>	<code>LocalDateTime</code>	When the event occurred
<code>getAction()</code>	<code>String</code>	The action identifier (e.g., REGISTERED)
<code>getDetails()</code>	<code>Map<String, Object></code>	Additional event context

Each concrete event class below adds its own service-specific fields on top of this common interface.

7.1.2 AuthEvent (User Service)

Collection: `auth_events`

Field	Type	Constraints	Notes
<code>id</code>	<code>String</code>	auto-generated	MongoDB ObjectId
<code>userId</code>	<code>Long</code>	not null	References User in PG
<code>action</code>	<code>String</code>	not null	See action values below
<code>timestamp</code>	<code>LocalDateTime</code>	not null	When the event occurred
<code>details</code>	<code>Map<String, Object></code>		Additional event context

action primary values: REGISTERED, LOGGED_IN, ROLE_CHANGED. **Non-exhaustive** — extend with domain-appropriate values when the Observer retrofit (Section 4.5) writes events from M1 user endpoints, e.g., USER_UPDATED (S1-F2 preferences), USER_DEACTIVATED (S1-F4), DEFAULT_ADDRESS_SET (S1-F7), USER_CREATED / USER_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.3 ProductEvent (Product Service)

Collection: `product_events`

Field	Type	Constraints	Notes
<code>id</code>	<code>String</code>	auto-generated	MongoDB ObjectId
<code>productId</code>	<code>Long</code>	not null	References Product in PG
<code>action</code>	<code>String</code>	not null	See action values below
<code>timestamp</code>	<code>LocalDateTime</code>	not null	
<code>details</code>	<code>Map<String, Object></code>		Fields changed, dashboard params, etc.

action primary values: INDEXED, DASHBOARD_VIEWED. **Non-exhaustive** — extend for M1 retrofits, e.g., SPECIFICATIONS_UPDATED (S2-F2), STATUS_CHANGED (S2-F4 discontinue), REVIEW_ADDED (S2-F7), REVIEW_VERIFIED (S2-F8), PRODUCT_CREATED / PRODUCT_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.4 OrderEvent (Order Service)

Collection: order_events

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
orderId	Long	not null	References Order in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
details	Map<String, Object>		

action primary values: ANALYTICS_VIEWED, INTERACTION_RECORDED. **Non-exhaustive** — extend for M1 retrofits, e.g., ORDER_CONFIRMED (S3-F2), ORDER_DELIVERED (S3-F4), ORDER_CANCELLED (S3-F7), ITEMS_ADDED (S3-F8), ORDER_CREATED / ORDER_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.5 ShipmentEvent (Shipping Service)

Collection: shipment_events

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
shipmentId	Long	not null	References Shipment in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
details	Map<String, Object>		

action primary values: TRACKING_RECORDED, ANALYTICS_VIEWED. **Non-exhaustive** — extend for M1 retrofits, e.g., SHIPMENT_CREATED (S4-F2), BATCH_STATUS_UPDATED (S4-F4), OLD_DATA_PURGED (S4-F7), SHIPMENT_DELETED (CRUD). Use UPPER_SNAKE_CASE.

7.1.6 TransactionAuditEvent (Billing Service)

Collection: transaction_audit_trail

Field	Type	Constraints	Notes
id	String	auto-generated	MongoDB ObjectId
transactionId	Long	not null	References Transaction in PG
action	String	not null	See action values below
timestamp	LocalDateTime	not null	
method	String	conditional	See method/amount note below
amount	Double	conditional	See method/amount note below
details	Map<String, Object>		Refund reason, refunded items, strategy name, etc.

action primary values: CREATED, COMPLETED, FAILED, REFUNDED, REFUND_DENIED, ANALYTICS_VIEWED. **Non-exhaustive** — extend for M1 retrofits, e.g., VOUCHER_APPLIED (S5-F5), RETRY_ATTEMPTED (S5-F7), TRANSACTION_DELETED (CRUD). Use UPPER_SNAKE_CASE.

method values: CREDIT_CARD, CASH_ON_DELIVERY, WALLET — matches the M1 Transaction.method enum values.

When each action value is written:

- **CREATED** — written by M1 S5-F4 (Process Transaction) when the Transaction row is first populated from PENDING (the PENDING row was created at S3-F4 Mark Order as Delivered; S5-F4 fills in the method and completes it).
- **COMPLETED** — written by M1 S5-F4 when the transaction status transitions to COMPLETED.

- **FAILED** — written by M1 S5-F4 when a simulated gateway failure occurs. Since M1 does not model a real payment gateway, enable failure simulation via an optional query parameter `?simulateFailure=true` on the S5-F4 process-transaction request: when set, S5-F4 short-circuits to `Transaction.status = FAILED` in PG and writes a **FAILED** audit event. This affordance exists solely to let the S5-F11 test scenario (and any Strategy denial-path test) produce failed-transaction data; it is not a production flow.
- **REFUNDED** — written by M1 S5-F2 (simple refund) and by M2 S5-F12 (partial item refund) when the refund is successfully processed.
- **REFUND_DENIED** — written by M2 S5-F12 when `NoRefundStrategy` blocks the refund (e.g., the return window has expired). The event records the denial reason and strategy name in `details`.
- **ANALYTICS_VIEWED** — written by M2 S5-F10 (Category Revenue with Return Impact) each time the endpoint is invoked, regardless of whether the response is served from cache. Used to track analytics access patterns.

method and amount field requirement: The fields `method` and `amount` are **required (not null)** on every event whose `action` is a payment-shaped action, i.e., `action ∈ {CREATED, COMPLETED, FAILED, REFUNDED, REFUND_DENIED}` and any transaction-lifecycle retrofit action (e.g., `VOUCHER_APPLIED, RETRY_ATTEMPTED`). They are **omitted (null-permitted)** only on non-payment actions such as `ANALYTICS_VIEWED` (an analytics view is not tied to a specific Transaction row’s `method/amount`). S5-F11 (Transaction Lifecycle Audit) relies on this consistency: it groups and filters only events whose `transactionId` matches, and populates timeline entries using the event’s `method` and `amount` when those fields are present.

7.2 Elasticsearch Document (Product Service)

7.2.1 ProductSearchDocument

Index: products

Field	ES Field Type	Purpose
<code>id</code>	Keyword	Exact match, corresponds to PG <code>id</code>
<code>name</code>	Text (analyzed)	Full-text search on product name
<code>description</code>	Text (analyzed)	Full-text search on product description (native M1 column)
<code>category</code>	Keyword	Filtering (free-form string in M1, e.g., <code>ELECTRONICS, CLOTHING, BOOKS</code>)
<code>brand</code>	Keyword	Filtering by brand
<code>price</code>	Double	Range filtering
<code>stockQuantity</code>	Integer	Range filtering
<code>rating</code>	Double	Range filtering
<code>status</code>	Keyword	Filtering (<code>ACTIVE, INACTIVE, OUT_OF_STOCK</code> — matches M1 <code>Product.status</code> enum)

Note on description: Unlike some M2 themes where the S2 entity’s `description` is additive, Amazon’s M1 `Product` entity **already declares** a `description` column as a native string (not JSONB). S2-F11 simply reads that column directly; no M1 schema change is needed. S2-F10 full-text search uses this column alongside `name` for relevance ranking.

7.3 Neo4j Entities (Order Service)

Amazon’s recommendation graph uses **product-to-product co-purchase edges**, which directly support “Customers who bought X also bought Y” queries without requiring a user-similarity traversal. There are no User nodes in this graph.

7.3.1 ProductNode

Property	Type	Notes
productId	Long	Corresponds to Product.id in PG
name	String	Product's display name
category	String	For filtering recommendations by category if needed

7.3.2 BOUGHT_TOGETHER Relationship

Connects two ProductNodes. Represented as a directed edge in Neo4j, but conceptually undirected. **Canonical direction: always create the edge from the product with the lower productId to the higher productId, and never create both directions.** Recommendation queries (S3-F12) traverse the edge with Cypher's undirected match operator so direction is transparent to the query layer. When an Order is delivered with ≥ 2 OrderItems, every unordered pair of products in that order gets a BOUGHT_TOGETHER edge created or incremented between them.

Property	Type	Notes
coPurchaseCount	Integer	Incremented each time the two products appear together in a delivered order
lastCoPurchaseDate	LocalDateTime	Updated on each increment
recorded_order_ids	List<Long>	Idempotency marker — see S3-F11 step d

Note on ProductNode staleness: ProductNodes are created on first co-purchase; their **name** and **category** properties are advisory snapshots. The authoritative product data is always re-read from PG on recommendation reads (S3-F12 step d), so user-visible DTOs are never stale even if the node itself is.

7.4 Cassandra Entity (Shipping Service)

7.4.1 ShipmentTrackingEvent

Table: shipment_tracking_events

Column	Type	Key	Notes
shipment_id	Long	Partition Key	Groups all events for one shipment
timestamp	Timestamp	Clustering (DESC)	Orders events newest-first
status	String		PROCESSING, SHIPPED, IN_TRANSIT, OUT_FOR_DELIVERY, DELIVERED, RETURNED (matches M1 Shipment.status)
carrier	String		Carrier name (matches M1 Shipment.carrier — M1 does not model carriers as a separate entity)
tracking_number	String		Snapshot of M1 Shipment.trackingNumber at the time of the event
latitude	Double		GPS latitude at the event time
longitude	Double		GPS longitude at the event time
notes	String		Optional event notes

Queries on this table **must** include `shipment_id` in the WHERE clause (partition key requirement).

8 Caching Strategy

All read-heavy endpoints (both new M2 features and existing M1 endpoints) must be cached in Redis.

8.1 TTL Guidelines

Data Type	TTL	Examples
Dashboards / analytics	10 minutes	Catalog dashboard, order analytics, shipping analytics, category revenue
Search results	5 minutes	Product full-text search, co-purchase recommendations
Activity feeds	5 minutes	User activity feed
Entity detail views	15 minutes	Product detail, order detail

8.2 Invalidation

Write operations (POST, PUT, DELETE) must invalidate any cached data they affect. The canonical invalidation enumeration for all M1 write endpoints + CRUD writes + M2 observer writes lives in Section 4.4.4 and uses the wildcard-deletion strategy described in Section 4.4.6. Features that mention invalidation (e.g., S5-F12 step h) refer to those rules. In brief:

- Entity write on `<entity>{id}` → delete `<service>::<entity>::<id>` plus wildcard-delete `<service>::S{n}-F{m}::*` for every feature whose cached output may reference that entity.
- M2 analytics events written by Observer retrofit (see Section 4.4.4 M2-writes subsection) also invalidate the matching dashboard caches — except for pure-observability actions (`ANALYTICS_VIEWED`, `DASHBOARD_VIEWED`).
- Over-invalidation (clearing a few cache keys that did not strictly need to be cleared) is acceptable — correctness beats cache-hit optimality at M2.

9 Cross-Cutting Requirements

These requirements apply globally and are **not** counted as numbered features. Each requirement has a dedicated test scenario below.

9.1 CC-1 — JWT on All Endpoints

Requirement: Every endpoint (including your M1 endpoints) must require a valid JWT token in the `Authorization` header, except `POST /api/auth/register`, `POST /api/auth/login`, and health checks. Missing or invalid token returns **401**. Insufficient role returns **403**.

Test scenario:

- Enumerate every endpoint in all 5 services by scanning controllers. For each: call without `Authorization` header → expect 401 unless the endpoint is `/api/auth/register`, `/api/auth/login`, or a health check.
- For each public endpoint: call without a token → expect a 2xx response (not 401).
- Call any protected endpoint with a malformed token (`Bearer abc`) → 401.
- Call any protected endpoint with an expired token → 401.
- Call `PUT /api/users/{id}/role` with a valid CUSTOMER token → 403.
- Count the protected vs public endpoints. Amazon should have exactly 7 public endpoints (register, login, and the 5 per-service health endpoints: `GET /api/users/health`, `GET /api/products/health`, `GET /api/orders/health`, `GET /api/shipments/health`, `GET /api/transactions/health`). Anything else public is a grading failure.

9.2 CC-2 — Role Management

Requirement: Implement `PUT /api/users/{id}/role` as an ADMIN-only endpoint that changes a user's role. Must log a `ROLE_CHANGED` event to `auth_events`.

Request body: `{"role": "ADMIN"}` (or `"CUSTOMER"`).

Behavior:

- Validate JWT and `role` claim is ADMIN — *throws 403 if not ADMIN*.
- Find user by ID — *throws 404 if user not found*.

- c) Validate requested role is a valid ENUM value (CUSTOMER or ADMIN) — *throws 400 if invalid*.
- d) Update the user's role and save.
- e) Log `ROLE_CHANGED` to `auth_events` with the old and new role in `details`.
- f) Invalidate cached user detail. **Note:** The `ROLE_CHANGED` write to `auth_events` automatically triggers Section 4.4.4 invalidation of `user-service::S1-F12::*` (the activity feed) via the Observer chain; no extra code is needed in this endpoint.
- g) Return the updated user with status *code 200*.

Token staleness after role change (accepted limitation): The target user's existing JWT continues to carry its **old role** claim until it expires. Since tokens have a 24-hour lifetime (see Section 5.2) and the spec does not require a token-revocation list, a demoted user (ADMIN → CUSTOMER) retains ADMIN authorization for up to 24 hours, and a promoted user (CUSTOMER → ADMIN) cannot exercise new ADMIN privileges until they log in again. This is an accepted limitation for M2. The `ROLE_CHANGED` audit event and the invalidated user-detail cache give operators the information they need, but do not revoke the in-flight JWT. Production-grade solutions (token revocation list, shorter expiry + refresh tokens, per-request PG role re-check) are out of scope for M2.

Test scenario:

- a) Seed an ADMIN and a CUSTOMER. Log in as ADMIN.
- b) `PUT /api/users/{customerId}/role` with body `{"role":"ADMIN"}` → 200, the target user's role is now ADMIN in PG.
- c) Verify the `auth_events` collection has a `ROLE_CHANGED` event with `details` containing the old and new roles.
- d) Verify the cached user detail (`user-service::user::{customerId}`) was removed from Redis.
- e) Verify cached keys under `user-service::S1-F12::{customerId}::*` were removed from Redis after the `ROLE_CHANGED` write propagated through the Observer chain (Section 4.4.4 NoSQL-writer invalidation).
- f) Log in as a different CUSTOMER and call `PUT /api/users/1/role` → 403.
- g) Call `PUT /api/users/999/role` with ADMIN token → 404.
- h) Call with body `{"role":"BANANA"}` → 400.
- i) Call without Authorization → 401.

9.3 CC-3 — Redis Caching on M1 Endpoints

Requirement: Your M1 feature GET endpoints (27 for Amazon — see Section 4.4.1) and CRUD GET `/api/<entity>/<id>` endpoints (10 for Amazon) must be cached per Section 4.4. List endpoints are **not** cached.

Test scenario: See Section 4.4.6 for the comprehensive cache test scenario. Additional spot checks:

- a) For each of the 27 cached M1 feature endpoints and 10 CRUD GET-by-ID endpoints: call twice, inspect Redis between calls, confirm a key with the expected `<service>::<featureId>::<param-hash>` or `<service>::<entity>::<id>` format exists.
- b) For every M1 write endpoint (the 18 feature writes + 30 CRUD writes): trigger the write, then query Redis and verify the affected detail cache key is gone and relevant feature-result caches are invalidated.
- c) Confirm `GET /api/<entity>` (list) endpoints produce NO cache entry.

9.4 CC-4 — Design Pattern Implementation

Requirement: The 7 design patterns defined in Section 3 must be implemented at their specified locations.

Test scenario: See each pattern's own test scenario in Section 3.2 through 3.8.

9.5 CC-5 — Docker Compose with 6 Databases

Requirement: Your `docker-compose.yaml` must include all 6 database containers (PostgreSQL, MongoDB, Redis, Elasticsearch, Neo4j, Cassandra) with versions matching Task 2 alongside your 5 application services.

Test scenario:

- a) Parse the team's `docker-compose.yaml`. Assert services named `postgres`, `mongo`, `redis`, `elasticsearch`, `neo4j`, `cassandra` exist.
- b) Assert image tags: `postgres:17`, `elasticsearch:8.19.12`, and `mongo:latest/redis:latest/neo4j:latest/cassandra:latest`.
- c) Assert memory caps: Redis command includes `-maxmemory 256mb -maxmemory-policy allkeys-lru`; Elasticsearch env has `ES_JAVA_OPTS=-Xms512m -Xmx512m`; Cassandra env has `MAX_HEAP_SIZE: 512M`; Neo4j env has `NEO4J_server_memory_heap_max__size: 512m`.
- d) Assert healthchecks exist for all databases.
- e) Run `docker compose up -d` → all 6 databases reach healthy state within 120 seconds.
- f) All 5 application services start successfully and connect to their databases.
- g) Total stack memory usage (`docker stats`) stays under 5 GB.

9.6 CC-6 — Application Configuration (application.yml)

Requirement: Each service's `application.yml` must include connection settings for the databases it uses, plus the shared JWT secret and expiration. Must be YAML format, not properties.

Test scenario:

- a) Assert every service has an `application.yml` file (not `application.properties`).
- b) For `user-service`, `product-service`, `order-service`, `shipping-service`, `billing-service`: assert `spring.datasource.url` is present and points to `postgres:5432/amazondb`.
- c) Assert `spring.data.mongodb.uri`, `spring.data.redis.host`, and `jwt.secret/jwt.expiration` are present in every service.
- d) `Product-service`: assert `spring.elasticsearch.uris` is present.
- e) `Order-service`: assert `spring.data.neo4j.uri` is present.
- f) `Shipping-service`: assert `spring.cassandra.contact-points` and `keyspace-name` are present.
- g) Boot each service in isolation with other services down → must start as long as PostgreSQL is up (hard dep). NoSQL unavailability must not prevent startup.

10 Features

Milestone 2 adds **15 new features** (3 per service). Feature numbering continues from M1 (which used F1–F9), so M2 features are **F10–F12** per service.

Every M2 feature touches at least 2 databases. Each feature specification lists which databases are involved, whether authentication is required, and the caching behavior.

Important Note: *Do not forget to apply the Layered architecture (having repository, service and controller) as the test case will test that each feature is implemented according to the layered architecture as explained to you in the Labs.*

10.1 User Service Features (port 8081)

10.1.1 [S1-F10] Register User

Branch: feat/user/S1-F10/<ID>

Endpoint: POST /api/auth/register

Auth: None (public endpoint)

Databases: PostgreSQL, MongoDB

Request body:

```
{
  "name": "Ahmed Hassan",
  "email": "ahmed@example.com",
  "password": "securePassword123",
  "phone": "+201012345678"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresIn": 86400000
}
```

Behavior:

- Validate that name, email, password, and phone are not blank –*throws 400 if any field is missing or blank*.
- Check that no user with this email or phone already exists –*throws 409 if the email or phone is already registered* (the M1 User entity requires both email and phone to be unique).
- Hash the password before saving with **BCrypt** (the stored password must not be the plaintext value).
- Create the user in PostgreSQL with role **CUSTOMER** (Amazon’s M1 default role — never assign **ADMIN** on register) and status **ACTIVE**.
- Log a **REGISTERED** event to the **auth_events** collection in MongoDB with the user’s ID and the current timestamp.
- Generate a token containing the user’s email (as the **sub** claim), the user’s numeric **User.id** (as a custom **uid** claim), and the user’s role (as a **role** claim). The **uid** claim is required by the ownership check in §10 (S1-F12); see Section 5.2 for the full payload contract.
- Return the token and expiration with status *code 201*.

Test scenario:

- a) POST /api/auth/register with valid data → 201 with token. Verify user exists in database.
- b) POST /api/auth/register with the same email → 409.
- c) POST /api/auth/register with the same phone (but different email) → 409.
- d) POST /api/auth/register with blank email or blank phone → 400.
- e) Verify the stored password is **not** the plaintext password (it should be a BCrypt hash).

10.1.2 [S1-F11] Login

Branch: feat/user/S1-F11/<ID>

Endpoint: POST /api/auth/login

Auth: None (public endpoint)

Databases: PostgreSQL, MongoDB

Request body:

```
{
  "email": "ahmed@example.com",
  "password": "securePassword123"
}
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "expiresIn": 86400000
}
```

Behavior:

- a) Find user by email in PostgreSQL *–throws 401 if user not found.*
- b) Verify the provided password against the stored hash *–throws 401 if the password does not match.*
- c) Log a LOGGED_IN event to the auth_events collection in MongoDB with the user’s ID and timestamp.
- d) Generate a token containing the user’s email (as the sub claim), the user’s numeric User.id (as a custom uid claim), and the user’s role (as a role claim). The uid claim is required by the ownership check in §10 (S1-F12); see Section 5.2 for the full payload contract.
- e) Return the token and expiration with status *code 200.*

Note on 401 for both error cases: Returning 401 for both “user not found” and “wrong password” is intentional — it prevents account enumeration. A caller cannot distinguish whether a given email is registered. Do **not** return 404 for missing email.

Test scenario:

- a) Register a user, then POST /api/auth/login with correct credentials → 200 with token.
- b) POST /api/auth/login with wrong password → 401.
- c) POST /api/auth/login with non-existent email → 401 (not 404 — this is deliberate to prevent account enumeration).
- d) Use the returned token to access a protected endpoint (e.g., GET /api/users/1) → should succeed. Access the same endpoint without a token → 401.

10.1.3 [S1-F12] Get User Activity Feed

Branch: feat/user/S1-F12/<ID>

Endpoint: GET /api/users/{id}/activity?page={page}&size={size}

Auth: Required (USER)

Databases: MongoDB, PostgreSQL, Redis

Response: Paginated list of activity events:

```
{
  "content": [
    {
      "action": "LOGGED_IN",
      "timestamp": "2026-04-20T14:30:00",
      "details": {}
    },
    {
      "action": "REGISTERED",
      "timestamp": "2026-04-20T14:25:00",
      "details": {"email": "ahmed@example.com"}
    }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 2
}
```

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid.*
- b) **Ownership check:** Verify the authenticated caller's uid claim from the JWT equals the path {id}, OR the caller's role claim is ADMIN –*throws 403 if the caller is neither the target user nor an ADMIN* (prevents account enumeration and PII leakage). The comparison is a direct numeric equality check on the JWT's uid claim — no PG lookup needed.
- c) Find user by ID in PostgreSQL –*throws 404 if user not found.*
- d) Query the auth_events collection in MongoDB for events belonging to this user, sorted by most recent first, paginated by the page and size parameters. **Defaults and bounds:** if page is omitted, default to 0; if size is omitted, default to 10; cap size at 100 — requests with size > 100 are silently clamped to 100.
- e) The response should be cached for 5 minutes. Subsequent identical requests within the cache window should return the cached version.
- f) Return the paginated activity feed with status *code 200.*

Note: The activity feed should also include events from role management operations. The role management endpoint (PUT /api/users/{id}/role, see Section 9 cross-cutting requirement CC-2) must also be implemented, and ROLE_CHANGED events should appear in this feed.

Test scenario:

- a) Register a user, login, then GET /api/users/{id}/activity with the user's own token → should contain REGISTERED and LOGGED_IN events ordered by most recent first.
- b) Using the seeded ADMIN token, trigger a role change via PUT /api/users/{id}/role changing the user's role from CUSTOMER to ADMIN. Then GET /api/users/{id}/activity with the affected user's token → should contain a ROLE_CHANGED event as the most recent entry.
- c) Register user A and user B. With user A's token, call GET /api/users/{B.id}/activity → 403 (ownership violation, not 404 — A's token is valid and B exists).

- d) With the seeded ADMIN token, call `GET /api/users/{any-user-id}/activity` → 200 (ADMIN bypasses ownership check).
- e) `GET /api/users/{id}/activity` without Authorization header → 401.
- f) `GET /api/users/999/activity` with an ADMIN token → 404 (ADMIN passes ownership, user not found).
- g) Verify pagination: create multiple events, request with `page=0&size=1` (with matching-owner token) → should return 1 event with correct total count.

10.2 Product Service Features (port 8082)

10.2.1 [S2-F10] Full-Text Product Search

Branch: `feat/product/S2-F10/<ID>`

Endpoint:

`GET /api/products/search/full-text?`

`query={text}&category={c}&brand={b}&status={s}&minPrice={n}&maxPrice={n}&minRating={n}&maxRating={n}`

Auth: Required (USER)

Databases: Elasticsearch, PostgreSQL, Redis

Response: List of matching products with relevance-based ordering.

Note on path: This is a **distinct endpoint** from M1's S2-F1 (`GET /api/products/search`). M1's endpoint searches PostgreSQL by category and price range and is unchanged; M2's new `/full-text` endpoint searches Elasticsearch on name and description. Both endpoints must coexist.

Behavior:

- a) Validate the JWT token *—throws 401 if missing or invalid.*
- b) Search products using full-text search on the `query` parameter, matching against the product's `name` and `description` (native M1 column). The search should handle partial matches and be case-insensitive.
- c) Optionally filter by `category` (exact match, free-form string e.g., `ELECTRONICS`, `CLOTHING`, `BOOKS`), `brand` (exact match), `status` (exact match, one of `ACTIVE`, `INACTIVE`, `OUT_OF_STOCK`), `minPrice`/`maxPrice` (range filter), and `minRating`/`maxRating` (range filter). All filter parameters are optional — if not provided, they are ignored.
- d) Results should be sorted by relevance to the search query.
- e) The response should be cached for 5 minutes.
- f) Return the matching products with status *code 200*. Return an empty list if no matches.

Test scenario:

- a) Create 3 products via CRUD and index them. Search with `query=phone` → should return products with “phone” in name or description.
- b) Search with `query=phone&category=ELECTRONICS` → should return only electronics products matching “phone”.
- c) Search with `query=phone&status=ACTIVE` → should return only ACTIVE matching products.
- d) Search with `query=phone&minPrice=500&maxPrice=2000` → should return only products whose price falls within the range.
- e) Search with `query=phone&minRating=4.0` → should return only highly-rated matching products.
- f) Search with `query=nonexistent` → empty list.
- g) Search without token → 401.

10.2.2 [S2-F11] Index Product for Search

Branch: feat/product/S2-F11/<ID>

Endpoint: POST /api/products/{id}/index

Auth: Required (USER)

Databases: PostgreSQL, Elasticsearch, MongoDB

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid.*
- b) Find product by ID in PostgreSQL –*throws 404 if product not found.*
- c) Create or update the product's search document in Elasticsearch with the product's id, name, description, category, brand, price, stockQuantity, rating, and status.
- d) Log an INDEXED event to the product_events collection in MongoDB via the Observer chain. The event's details object must include productId, the list of indexedFields written in the prior step, and a source key: "explicit" when invoked through this endpoint, "auto_crud_create" / "auto_crud_update" when triggered by the CRUD auto-index retrofit. On DELETE the retrofit emits PRODUCT_DELETED (not INDEXED) after removing the ES document.
- e) **Auto-index on CRUD changes:** In addition to this explicit endpoint, the product must be automatically re-indexed whenever it is created or updated via any M1 CRUD endpoint, and removed from the search index whenever it is deleted via DELETE /api/products/{id}. This is graded — tests will create/update a product via CRUD and then search for it without calling this endpoint explicitly; after a delete, searching for the deleted product must return no match.
- f) Return status *code 200* indicating successful indexing.

Test scenario:

- a) Create a product via CRUD with description='4K OLED smart television', then POST /api/products/{id}/index → 200. Search with query=OLED → should return the product. Verify an INDEXED document exists in product_events with details.source="explicit".
- b) Create a product via CRUD (no explicit index call) → the auto-index retrofit writes an INDEXED document with details.source="auto_crud_create" and the product is searchable.
- c) Update a product's name via CRUD (without calling /index) → searching with the new name finds the updated product; verify an INDEXED document with details.source="auto_crud_update" was written.
- d) Delete a product via DELETE /api/products/{id} → the ES document is removed, and a PRODUCT_DELETED event (not INDEXED) appears in product_events. Searching for the deleted product returns no matches.
- e) POST /api/products/999/index → 404.
- f) Index without token → 401.

10.2.3 [S2-F12] Get Product Catalog Dashboard

Branch: feat/product/S2-F12/<ID>

Endpoint: GET /api/products/catalog/dashboard

Auth: Required (USER)

Databases: PostgreSQL, MongoDB, Redis

Response DTO (ProductCatalogDashboardDTO): totalProducts, outOfStockCount, averageRating, categoryDistribution (map of category to count), averagePrice, lowStockCount (products with stockQuantity ≤ 10).

Behavior:

- a) Validate the JWT token —*throws 401 if missing or invalid*.
- b) Aggregate from the `products` table:
 - `totalProducts` = count of all products regardless of status.
 - `outOfStockCount` = count of products whose `status` = `OUT_OF_STOCK`.
 - `averageRating` = average of `rating` across products with `totalRatings` > 0; return 0 if no rated products exist.
 - `categoryDistribution` = count of products grouped by `category`.
 - `averagePrice` = average of `price` across all products.
 - `lowStockCount` = count of products whose `stockQuantity` ≤ 10 and `status` = `ACTIVE` (matches the M1 S2-F9 low-stock threshold convention).
- c) Log a `DASHBOARD_VIEWED` event to the `product_events` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Section 4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.
- d) The response should be cached for 10 minutes.
- e) Return the dashboard DTO with status *code 200*.

Test scenario:

- a) Create 6 products: 3 `ELECTRONICS` (1 `OUT_OF_STOCK` with rating 4.5, 2 `ACTIVE` with ratings 4.0 and 3.5), 2 `CLOTHING` (both `ACTIVE`, ratings 4.2 and 4.8), 1 `BOOKS` (`ACTIVE`, unrated). `GET /api/products/catalog/dashboard` → `totalProducts`=6, `outOfStockCount`=1, `averageRating` ≈ 4.2, `categoryDistribution`=`{ELECTRONICS:3, CLOTHING:2, BOOKS:1}`.
- b) Empty catalog → `totalProducts`=0, `outOfStockCount`=0, `averageRating`=0, `categoryDistribution` empty.
- c) Without token → 401.
- d) Verify every invocation (including cache hits) writes a `DASHBOARD_VIEWED` event to `product_events`.

10.3 Order Service Features (port 8083)

10.3.1 [S3-F10] Get Order Analytics Dashboard

Branch: `feat/order/S3-F10/<ID>`

Endpoint: `GET /api/orders/analytics/dashboard?startDate={date}&endDate={date}`

Auth: Required (USER)

Databases: PostgreSQL, MongoDB, Redis

Response DTO (`OrderAnalyticsDashboardDTO`): `totalOrders`, `totalRevenue`, `averageOrderValue`, `completionRate`, `ordersByStatus` (map of status to count).

Note on path and DTO: This is a **distinct endpoint and DTO** from M1's S3-F6. M1's `GET /api/orders/analytics` returns `OrderAnalyticsDTO` (`totalOrders`, `deliveredOrders`, `cancelledOrders`, `totalRevenue`, `averageOrderValue`, `completionRate`) and remains unchanged. M2's new `/analytics/dashboard` endpoint returns the richer `OrderAnalyticsDashboardDTO` with a full status breakdown map. Both must coexist — do not overwrite either.

Behavior:

- a) Validate the JWT token —*throws 401 if missing or invalid*.

- b) Validate date range —*throws 400 if startDate is after endDate*. Both `startDate` and `endDate` are `LocalDate` values on the wire; internally expand the filter window to the fully-closed range [`startDate` T00:00:00, `endDate` T23:59:59.999] in the server time zone, matching the PG `TIMESTAMP` columns (`orderedAt`, `createdAt`, etc.) and the MongoDB `timestamp` field. Boundary rows (events exactly at `startDate` T00:00:00 or `endDate` T23:59:59.999) are included.
- c) Aggregate order statistics from the `orders` table for the given date range (filtered on `orderedAt`): total orders, total revenue (sum of `totalAmount` across all orders whose `totalAmount` is non-null), average order value, completion rate (DELIVERED orders divided by total orders), and a breakdown of order counts by status (PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, RETURNED).
- d) Log an `ANALYTICS_VIEWED` event to the `order_events` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Section 4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.
- e) The response should be cached for 10 minutes.
- f) Return the analytics DTO with status *code 200*.

Test scenario:

- a) Create 10 orders in March 2026: 6 DELIVERED (total 3000), 2 CANCELLED, 1 RETURNED, 1 PENDING. GET `/api/orders/analytics/dashboard?startDate=2026-03-01&endDate=2026-03-31` → `totalOrders=10, completionRate=0.6, ordersByStatus={DELIVERED:6, CANCELLED:2, RETURNED:1, PENDING:1}`.
- b) Query with dates where no orders exist → all values 0.
- c) `startDate=2026-04-01&endDate=2026-03-01` → 400.
- d) Without token → 401.

10.3.2 [S3-F11] Record Product Co-Purchase

Branch: `feat/order/S3-F11/<ID>`

Endpoint: `POST /api/orders/{orderId}/record-co-purchase`

Auth: Required (USER)

Databases: Neo4j, PostgreSQL, MongoDB

Behavior:

- a) Validate the JWT token —*throws 401 if missing or invalid*.
- b) Find the order by ID in PostgreSQL —*throws 404 if order not found*.
- c) Verify the order's status is `DELIVERED` —*throws 400 if the order is not delivered* (we only record co-purchase signals for completed orders).
- d) Load the order's `OrderItem` list. If the list contains fewer than 2 items, return 200 immediately without mutating the graph (a single-item order produces no co-purchase pairs).
- e) **Idempotency check:** Before mutating the graph, check whether this specific `orderId` has already been recorded. Maintain the idempotency marker **inside Neo4j** (not PostgreSQL — M2 does not alter any M1 PG schema per Section 1). Two acceptable Neo4j-only approaches: (a) each `BOUGHT_TOGETHER` relationship carries a `recorded_order_ids` collection property that accumulates the `orderIds` contributing to the edge's `coPurchaseCount` (the idempotency check passes (graph needs mutation) **only if** `orderId` is absent from **EVERY** edge this order would

touch; if `orderId` appears on any edge, treat the whole order as recorded and return 200 without further mutation. Step g must then atomically (in a single Cypher transaction) both increment `coPurchaseCount` and append `orderId` to `recorded_order_ids` for every pair, so partial-failure cannot leave an edge un-tagged.); or (b) a dedicated idempotency sentinel node, e.g., `(:RecordedOrder {orderId: ...})`, checked with an `EXISTS` query. Approach (b) with a dedicated sentinel node avoids this atomicity concern and is recommended for teams not comfortable with multi-edge Cypher transactions. If the marker says this order was already recorded, **return 200 immediately without mutating any edge**. This guarantees the endpoint is idempotent under repeat POSTs without introducing any M1 PostgreSQL schema change.

- f) For each product in the order, find or create its `ProductNode` in Neo4j (with `productId`, `name`, `category` looked up from the shared PostgreSQL `products` table using the cross-service native SQL pattern — **do not introduce inter-service HTTP calls**; those come in Milestone 3).
- g) For every unordered pair (`p_i`, `p_j`) of distinct `productIds` in the order's item list — deduplicate by `productId` before computing pairs; do not create self-loop edges (`P`, `P`) even when an order has multiple `OrderItems` with the same `productId`, find or create a `BOUGHT_TOGETHER` relationship between them. If the relationship already exists, increment `coPurchaseCount` by 1 and update `lastCoPurchaseDate`. If it is new, set `coPurchaseCount` to 1. Record this `orderId` in the idempotency marker from step e so future calls with the same `orderId` are no-ops.
- h) Log an `INTERACTION_RECORDED` event to the `order_events` collection in MongoDB via the Observer chain. The event's `details` object must include `orderId` and the list of `productIds` that participated in the co-purchase pairs. This write is what causes the S3-F12 recommendation cache to be wildcard-invalidated per Section 4.4.4 (NoSQL-writer → cached-reader rule); it is therefore required on the non-idempotent path only — on the idempotent short-circuit at step e (or the single-item short-circuit at step d), the graph did not change, so do **not** emit this event.
- i) Return status *code 200* with a confirmation message.

Test scenario:

- a) Create a delivered order O1 with two items: product P1 and product P2. `POST /api/orders/O1/record-co-purchase → 200`; verify a `BOUGHT_TOGETHER` edge exists between P1 and P2 with `coPurchaseCount=1`.
- b) Call the same endpoint again on the same O1 → 200, but `coPurchaseCount` is still 1 (idempotency — repeated calls on the same `orderId` must not inflate the counter).
- c) Create a second delivered order O2 with items P1 and P3. `POST /api/orders/O2/record-co-purchase → 200`; verify `BOUGHT_TOGETHER(P1, P3)` has `coPurchaseCount=1` and `BOUGHT_TOGETHER(P1, P2)` is still 1.
- d) Create a delivered order O3 with items P1, P2, P3 (three items → three pairs). `POST /api/orders/O3/record-co-purchase → 200`; verify `BOUGHT_TOGETHER(P1, P2)` increments to 2, `BOUGHT_TOGETHER(P1, P3)` increments to 2, and `BOUGHT_TOGETHER(P2, P3)` has `coPurchaseCount=1`.
- e) Call with a `PENDING` (not delivered) order → 400.
- f) Call with a delivered order that has only 1 item → 200 with no edge mutation (verify by Cypher that no new edges were created).
- g) Call with non-existent order ID → 404.
- h) Without token → 401.

10.3.3 [S3-F12] Get “Customers Also Bought” Recommendations

Branch: feat/order/S3-F12/<ID>

Endpoint: GET /api/orders/recommendations?productId={id}&limit={n}

Auth: Required (USER)

Databases: Neo4j, PostgreSQL, Redis

Response: List of ProductRecommendationDTO: productId, name, category, brand, price, score (the coPurchaseCount of the recommending edge).

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid*.
- b) Find the seed product by ID in PostgreSQL –*throws 404 if product not found*.
- c) Traverse the recommendation graph: starting from the seed product’s ProductNode, find all ProductNodes connected by a BOUGHT_TOGETHER relationship (in either direction). Rank them by the relationship’s coPurchaseCount, descending. Exclude the seed product itself from the result.
- d) Enrich the results with product details (name, category, brand, price) from PostgreSQL using the cross-service native SQL pattern. **Drop any candidate whose status $\neq$ ACTIVE** or whose row no longer exists in PG (deleted products or inactive/out-of-stock products must not appear in recommendations).
- e) Limit results to the top limit recommendations. Default limit to 5 if not provided.
- f) The response should be cached for 5 minutes.
- g) Return the recommendations with status *code 200*. Return an empty list if no recommendations can be made.

Test scenario:

- a) Create 4 products (P1, P2, P3, P4). Record co-purchases via S3-F11 so the graph has: BOUGHT_TOGETHER(P1, P2)=3, BOUGHT_TOGETHER(P1, P3)=1, BOUGHT_TOGETHER(P2, P4)=2. GET /api/orders/recommendations?productId=P1&limit=5 → should return [P2 with score 3, P3 with score 1]; P4 is not returned because it has no direct edge to P1.
- b) Same query with limit=1 → should return only P2.
- c) GET /api/orders/recommendations?productId=P4&limit=5 → should return [P2 with score 2].
- d) Query for a product with no recorded co-purchases → empty list.
- e) productId=999 → 404.
- f) Call the endpoint twice with identical parameters → second call served from cache (within 5 min); then call S3-F11 on a new delivered order involving the seed product → the cached recommendation key is wildcard-invalidated (Section 4.4.4), and the next GET recomputes.
- g) Without token → 401.

10.4 Shipping Service Features (port 8084)

10.4.1 [S4-F10] Get Shipping Analytics Dashboard

Branch: feat/shipping/S4-F10/<ID>

Endpoint: GET /api/shipments/analytics?startDate={date}&endDate={date}

Auth: Required (USER)

Databases: PostgreSQL, MongoDB, Redis

Response DTO (ShippingAnalyticsDTO): totalShipments, averageDeliveryTimeDays, onTimeRate, shipmentsByStatus (map of status to count), averageAttempts (average

metadata.deliveryAttempts across delivered shipments).

Behavior:

- a) Validate the JWT token *–throws 401 if missing or invalid.*
- b) Validate date range *–throws 400 if startDate is after endDate.* Both `startDate` and `endDate` are `LocalDate` values on the wire; internally expand the filter window to the fully-closed range `[startDate T00:00:00, endDate T23:59:59.999]` in the server time zone, matching the PG `TIMESTAMP` columns (`createdAt`, `lastUpdate`) and the `LocalDate` columns (`estimatedDelivery`, `actualDelivery`). Boundary rows (events exactly at `startDate T00:00:00` or `endDate T23:59:59.999`) are included.
- c) Aggregate from the `shipments` table. Filter on `createdAt` within the date range:
 - `totalShipments` = count of shipments in the range.
 - `averageDeliveryTimeDays` = average of `(actualDelivery - createdAt)` in days, across only `DELIVERED` shipments in the range whose `actualDelivery` is non-null. Return 0 if none delivered.
 - `onTimeRate` = count of delivered shipments with `actualDelivery ≤ estimatedDelivery` divided by the number of delivered shipments in the range. Return 0 if none delivered.
 - `shipmentsByStatus` = count of shipments grouped by their `status` column (`PROCESSING`, `SHIPPED`, `IN_TRANSIT`, `OUT_FOR_DELIVERY`, `DELIVERED`, `RETURNED`).
 - `averageAttempts` = average of `metadata->'deliveryAttempts'` cast to numeric, across `DELIVERED` shipments. Treat a missing or null key as 1. Return 0 if no delivered shipments.
- d) Log an `ANALYTICS_VIEWED` event to the `shipment_events` collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that `ANALYTICS_VIEWED` / `DASHBOARD_VIEWED` are pure-observability actions and are excluded from Observer-driven cache invalidation (see Section 4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.
- e) The response should be cached for 10 minutes.
- f) Return the analytics DTO with status *code 200*.

Test scenario:

- a) Create 8 shipments in April 2026: 5 `DELIVERED` (3 with `actualDelivery ≤ estimatedDelivery`, 2 later), 2 `IN_TRANSIT`, 1 `PROCESSING`. `GET /api/shipments/analytics?startDate=2026-04-01&endDate=2026-04-30` → `totalShipments=8`, `onTimeRate=0.6` (3 out of 5 on-time), `averageDeliveryTimeDays` computed from the 5 `DELIVERED` shipments, `shipmentsByStatus={DELIVERED:5, IN_TRANSIT:2, PROCESSING:1}`.
- b) Dates with no shipments → `totalShipments=0`, `averageDeliveryTimeDays=0`, `onTimeRate=0`, empty `shipmentsByStatus`.
- c) Invalid date range → 400.
- d) Without token → 401.

10.4.2 [S4-F11] Record Shipment Tracking Event

Branch: `feat/shipping/S4-F11/<ID>`

Endpoint: `POST /api/shipments/{id}/tracking`

Auth: Required (USER)

Databases: Cassandra, PostgreSQL, MongoDB

Request body:

```
{
  "status": "IN_TRANSIT",
  "latitude": 30.0444,
  "longitude": 31.2357,
  "notes": "Departed Cairo sorting facility"
}
```

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid.*
- b) Find shipment by ID in PostgreSQL –*throws 404 if shipment not found.*
- c) Store a new tracking event in the Cassandra time-series table `shipment_tracking_events` with `shipment_id` as the partition key, the current timestamp as the clustering column, and the status, carrier (from the `carrier` field on the shipment entity in PostgreSQL), `tracking_number` (from `trackingNumber`), latitude, longitude, and notes from the request. Cassandra is the **primary storage** for the tracking time-series.
- d) **Additionally**, fire the Observer chain to log a `TRACKING_RECORDED` event to the `shipment_events` MongoDB collection with the `shipmentId`, the status, and a summary of the tracking payload in **details**. Both writes must succeed independently — Cassandra for the time-series trail, MongoDB as the observability audit log that feeds the activity feed and the S5-F11 lifecycle audit.
- e) Return status *code 201* indicating the event was recorded.

Test scenario:

- a) Create a shipment via CRUD. `POST /api/shipments/{id}/tracking` with status `SHIPPED` → 201. Verify a row exists in Cassandra `shipment_tracking_events` for this `shipment_id`, AND a `TRACKING_RECORDED` document exists in MongoDB `shipment_events`.
- b) Record another event with status `IN_TRANSIT` → 201. Query the timeline (S4-F12) → should show both events in reverse chronological order.
- c) `POST /api/shipments/999/tracking` → 404.
- d) Without token → 401.

10.4.3 [S4-F12] Get Shipment Tracking Timeline

Branch: `feat/shipping/S4-F12/<ID>`

Endpoint: `GET /api/shipments/{id}/tracking?startTime={datetime}&endTime={datetime}`

Auth: Required (USER)

Databases: Cassandra, PostgreSQL, Redis

Response: List of `ShipmentTrackingDTO`: timestamp, status, carrier, `trackingNumber`, latitude, longitude, notes.

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid.*
- b) Find shipment by ID in PostgreSQL –*throws 404 if shipment not found.*
- c) Query the Cassandra time-series table for all tracking events for this shipment, optionally filtered by the `startTime` and `endTime` parameters (interpreted as `LocalDateTime` in the server time zone). If no time range is provided, return all events. Results are sorted by most recent first (Cassandra clustering order).
- d) The response should be cached for 5 minutes.

- e) Return the tracking timeline with status *code 200*. Return an empty list if no events exist.

Test scenario:

- a) Create a shipment. Record 3 tracking events: PROCESSING at 14:00, SHIPPED at 14:15, IN_TRANSIT at 14:30. GET /api/shipments/{id}/tracking → should return 3 events with IN_TRANSIT first.
- b) Query with startTime=2026-04-20T14:10:00&endTime=2026-04-20T14:20:00 → should return only the SHIPPED event.
- c) GET /api/shipments/999/tracking → 404.
- d) Shipment with no tracking events → empty list.
- e) Call the endpoint twice with identical parameters → second call served from cache (within 5 min); then call S4-F11 to record a new tracking event → the cached shipping-service::S4-F12::{shipmentId} key is invalidated (Section 4.4.4 NoSQL-writer rule), and the next GET recomputes and includes the new event.
- f) Without token → 401.

10.5 Billing Service Features (port 8085)

10.5.1 [S5-F10] Get Category Revenue with Return Impact

Branch: feat/billing/S5-F10/<ID>

Endpoint: GET /api/transactions/analytics/category?startDate={date}&endDate={date}

Auth: Required (USER)

Databases: PostgreSQL, MongoDB, Redis

Response: List of CategoryRevenueDT0: category, grossRevenue, refundedRevenue, netRevenue, transactionCount, refundCount, returnRate.

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid*.
- b) Validate date range –*throws 400 if startDate is after endDate*. Both startDate and endDate are LocalDate values on the wire; internally expand the filter window to the fully-closed range [startDate T00:00:00, endDate T23:59:59.999] in the server time zone, matching the PG TIMESTAMP columns (orderedAt, createdAt) and the MongoDB timestamp field. Boundary rows (events exactly at startDate T00:00:00 or endDate T23:59:59.999) are included.
- c) Use the M1 cross-service native SQL pattern: JOIN transactions with orders (on orders.id = transactions.order_id), order_items (on order_items.order_id = orders.id), and products (on products.id = order_items.product_id) on the shared PostgreSQL database. **Do not introduce inter-service HTTP calls**; those come in Milestone 3.
- d) Filter by orders.orderedAt within [startDate, endDate] and transactions.status ∈ {COMPLETED, REFUNDED}.
- e) Group by products.category. For each group compute:
 - grossRevenue = sum of (order_items.quantity * order_items.priceAtPurchase) across all items whose parent transaction is COMPLETED or REFUNDED. This represents total money charged at purchase time before refunds.

- **refundedRevenue** = sum of the actual refunded amount per transaction: for transactions with **transactionDetails.refundedItems** present (M2 S5-F12 writes this; see Section 4.6), sum the **amount** field across only the refunded items that belong to products in this category. For pre-M2 or S5-F2 full-refund transactions that lack **refundedItems**, fall back to summing **order_items.quantity * order_items.priceAtPurchase** across all items of those REFUNDED transactions in this category (treats the whole transaction as fully refunded). Document the mixed-precision note in the response DTO.
 - **netRevenue** = **grossRevenue** - **refundedRevenue**.
 - **transactionCount** = count of distinct transactions (any status in {COMPLETED, REFUNDED}) that contain at least one item in this category. **Per-category-local:** a transaction whose order spans N categories contributes to each of their **transactionCounts**. The sum of **transactionCount** across all categories may therefore exceed the global distinct transaction count.
 - **refundCount** = count of distinct REFUNDED transactions that contain at least one item in this category.
 - **returnRate** = **refundCount** / **transactionCount** (return 0 if denominator is 0).
- f) Log an **ANALYTICS_VIEWED** event to the **transaction_audit_trail** collection in MongoDB. **This log must be written on every invocation**, independently of whether the response was served from cache — perform the logging step **outside** the cache decorator/layer so it runs on cache hits too. Note that **ANALYTICS_VIEWED** / **DASHBOARD_VIEWED** are pure-observability actions and are excluded from Observer-driven cache invalidation (see Section 4.4.4) — they do not mutate the data that this dashboard reads, so invalidating the cache on them would defeat the cache.
- g) The response should be cached for 10 minutes.
- h) Return the breakdown with status **code 200**. Return an empty list if no data exists for the date range.

Test scenario:

- Create products: P1 (ELECTRONICS, price 500), P2 (ELECTRONICS, price 300), P3 (CLOTHING, price 150). Create 3 delivered orders and their completed transactions in March 2026: O1 with items [P1 × 1] (=500) transaction COMPLETED; O2 with items [P2 × 2] (=600) transaction COMPLETED; O3 with items [P3 × 1] (=150) transaction REFUNDED. GET **/api/transactions/analytics/category?startDate=2026-03-01&endDate=2026-03-31** → ELECTRONICS: **grossRevenue=1100, refundedRevenue=0, netRevenue=1100, transactionCount=2, refundCount=0, returnRate=0**. CLOTHING: **grossRevenue=150, refundedRevenue=150, netRevenue=0, transactionCount=1, refundCount=1, returnRate=1.0**.
- Create one PartialItem-refunded transaction via S5-F12 refunding \$50 of a \$500 ELECTRONICS transaction → verify ELECTRONICS.refundedRevenue reflects exactly \$50, not \$500.
- Date range with no transactions → empty list.
- Invalid date range → 400.
- Every invocation writes an **ANALYTICS_VIEWED** event to **transaction_audit_trail**, even on cache hits.
- Without token → 401.

10.5.2 [S5-F11] Get Transaction Lifecycle Audit

Branch: feat/billing/S5-F11/<ID>

Endpoint: GET **/api/transactions/{transactionId}/lifecycle**

Auth: Required (USER)

Databases: MongoDB, PostgreSQL, Redis

Response: List of LifecycleEventDTO: timestamp, source (one of TRANSACTION, ORDER, SHIPMENT), action (the MongoDB event's action string), details (pass-through).

Behavior:

- a) Validate the JWT token –*throws 401 if missing or invalid.*
- b) Find transaction by ID in PostgreSQL –*throws 404 if transaction not found.* Capture its orderId.
- c) **Ownership check:** Verify the authenticated caller's uid claim from the JWT equals the transaction's userId, OR the caller's role claim is ADMIN –*throws 403 if the caller is neither the transaction owner nor an ADMIN.* Prevents cross-user lifecycle enumeration.
- d) Using the cross-service native SQL pattern, list the shipment IDs whose order_id matches this transaction's orderId (a single order may have zero, one, or many shipments).
- e) Assemble a unified timeline by querying three MongoDB collections:
 - **transaction_audit_trail:** all events whose transactionId equals the path parameter. Tag each with source="TRANSACTION".
 - **order_events:** all events whose orderId equals the transaction's orderId. Tag each with source="ORDER".
 - **shipment_events:** all events whose shipmentId is in the shipment-ID list from step d. Tag each with source="SHIPMENT".

This naturally captures the payment → shipped → delivered → returned event chain the feature promises. Pure-observability entries (ANALYTICS_VIEWED, DASHBOARD_VIEWED) are included in the timeline since they are part of the audit record.

- f) Merge all three result sets into a single list and sort by timestamp ascending (chronological order).
- g) The response should be cached for 10 minutes.
- h) Return the timeline with status **code 200**. Return an empty list if no events exist for this transaction.

Test scenario:

- a) Seed a complete flow: create an order O1 with 2 items, create a shipment S1 for O1, then call M1 S3-F4 Mark Delivered, M1 S5-F4 Process Transaction (which writes CREATED + COMPLETED audit events), M1 S4-F2 Create Shipment (SHIPMENT_CREATED), S4-F11 Record Tracking (TRACKING_RECORDED). Call GET /api/transactions/{transactionId}/lifecycle → timeline contains TRANSACTION::CREATED, TRANSACTION::COMPLETED, ORDER::ORDER_DELIVERED, SHIPMENT::SHIPMENT_CREATED, SHIPMENT::TRACKING_RECORDED in chronological order.
- b) Refund the transaction via M1 S5-F2 → next call adds TRANSACTION::REFUNDED to the timeline.
- c) GET /api/transactions/999/lifecycle → 404.
- d) Call on a freshly-created transaction that has no shipments yet → returns only TRANSACTION events.
- e) Create transactions for user A and user B. With user A's token, call GET /api/transactions/{B.txId}/lifecycle → 403. With the seeded ADMIN token → 200.
- f) Without token → 401.

10.5.3 [S5-F12] Process Partial Item Refund

Branch: feat/billing/S5-F12/<ID>

Endpoint: POST /api/transactions/{id}/refund-items

Auth: Required (USER)

Databases: PostgreSQL, MongoDB, Redis

Design Pattern: Strategy (see Section 3.2)

Request body:

```
{
  "reason": "defective_items",
  "refundAll": false,
  "orderItemIds": [12, 14]
}
```

Note on path: This is a **distinct endpoint** from M1's S5-F2 (PUT /api/transactions/{id}/refund). M1's endpoint performs a simple full refund via a reason string and remains unchanged; M2's new /refund-items endpoint selects a refund strategy at runtime (see Section 3.2) and writes a richer MongoDB audit trail including the list of refunded items. Both must coexist.

Behavior:

- a) Validate the JWT token —*throws 401 if missing or invalid.*
- b) Find transaction by ID in PostgreSQL —*throws 404 if transaction not found.*
- c) Validate the transaction status is COMPLETED —*throws 400 if the transaction is not in COMPLETED status* (you cannot refund a transaction that has not been completed). **Single-refund-per-transaction contract:** Because step g sets the transaction status to REFUNDED, any subsequent call to this endpoint on the same transaction will fail at step c with 400. Each transaction is therefore refundable at most once, whether the first call used FullItemRefundStrategy or PartialItemRefundStrategy. Refunding additional items after a partial refund is out of scope for M2 — M3 will introduce a multi-refund contract.
- d) If refundAll=false, validate that orderItemIds is non-empty AND every id in orderItemIds belongs to the order linked by this transaction (transactions.order_id) —*throws 400 if the list is empty or any id does not belong to this transaction's order.* If refundAll=true, orderItemIds in the request body is ignored (the strategy refunds every item in the transaction's order regardless of whether orderItemIds is present or non-empty).
- e) Select the refund strategy (see Section 3.2 for the three required strategies) **without throwing yet:** FullItemRefundStrategy (when refundAll=true and the transaction is within the 30-day return window from createdAt), PartialItemRefundStrategy (when refundAll=false, orderItemIds is non-empty, and the transaction is within the 30-day return window), or NoRefundStrategy (when the transaction is older than 30 days from createdAt).
- f) **If the selected strategy is NoRefundStrategy:** (i) Log a REFUND_DENIED event to the transaction_audit_trail collection in MongoDB with the strategy name NoRefundStrategy, the denial reason "return window expired", the transaction's method and amount, and the requested orderItemIds in details. (ii) Invalidate billing-service::S5-F10::* and billing-service::S5-F11::* (per Section 4.4.4 NoSQL-writer rule) — even though no PostgreSQL row mutates on the denial path, the new REFUND_DENIED audit event changes what those analytics would return on the next read. (iii) Then *throw 400 with message "return window expired"*. Logging and cache invalidation must both happen **before** the throw so the audit event is persisted and cached reads pick up the new event immediately.
- g) Otherwise (strategy is FullItemRefundStrategy or PartialItemRefundStrategy): invoke the strategy's calculateRefund(transaction, request) to obtain the refund amount and the list of refunded orderItemIds (FullItemRefundStrategy returns every item of the transaction's order; PartialItemRefundStrategy returns only the requested ids). Update the transaction status to REFUNDED.

- h) Record the refund in the `transactionDetails` JSONB column of the Transaction (reusing the M1 JSONB pattern from S5-F2, additively per Section 4.6). Add keys: `refundAmount`, `refundedItems` (list of `{orderId, quantity, amount}` objects), `refundStrategy` (the concrete class name), `refundReason`, and `refundedAt`. Do **not** add a new SQL column.
- i) Log a REFUNDED event to the `transaction_audit_trail` collection in MongoDB with the strategy name, reason, original transaction amount, refund amount, and the list of refunded `orderId`s in `details`. Set the event's `method` and `amount` fields (required for payment-shaped actions; see Section 7.1.5).
- j) Invalidate `billing-service::S5-F10::*`, `billing-service::S5-F11::*`, and `billing-service::transaction:::{id}` (per Section 4.4.4 NoSQL-writer + PG-writer rules). Over-invalidation is acceptable per Section 8.2.
- k) Return the updated transaction with status *code 200*.

Test scenario:

- a) Create a completed transaction of 500 linked to an order with items [I1 (price 200×1), I2 (price 150×2)], `createdAt` today. POST `/api/transactions/{id}/refund-items` with `{"reason":"defective","refundAll":true}` → 200, refund amount = 500, status = REFUNDED. Verify `transactionDetails.refundAmount=500`, `transactionDetails.refundStrategy="FullItemRefundStrategy"`, and `refundedItems` lists both I1 and I2.
- b) Create another completed transaction of 500 with items [I3 (200), I4 (150), I5 (150)], `createdAt` today. Refund with `{"reason":"damaged","refundAll":false,"orderId": [I3.id]}` → 200, refund amount = 200. Verify `transactionDetails.refundAmount=200`, `transactionDetails.refundStrategy="PartialItemRefundStrategy"`, `refundedItems` contains only I3.
- c) Create a transaction with `createdAt` 45 days ago → POST `/api/transactions/{id}/refund-items` → 400 with message "return window expired" (NoRefundStrategy selected). Verify that a REFUND_DENIED document exists in `transaction_audit_trail` for this `transactionId` with `details.refundStrategy="NoRefundStrategy"` (audit log is written **before** the 400 is thrown). Also verify that any previously-cached keys under `billing-service::S5-F10::*` and `billing-service::S5-F11::*` were removed from Redis (cache invalidation on the denial path).
- d) Attempt to refund a PENDING transaction → 400.
- e) Attempt to refund an already-REFUNDED transaction → 400.
- f) Verify that after a PartialItemRefundStrategy refund of items [I1], a second call attempting to refund [I2] on the same `transactionId` returns 400 (single-refund-per-transaction contract).
- g) Call with `refundAll=false` and an empty `orderId`s list → 400.
- h) Call with `refundAll=false` and an `orderId` that belongs to a different order → 400.
- i) POST `/api/transactions/999/refund-items` → 404.
- j) Without token → 401.